AI-Based Paddy Seed Quality Assessment And Farmer Feedback System

A thesis submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

By

Joymangol Chingangbam	**(2201CS0004)**
Nilambar Elangbam	**(2201CS0005)**
Khangembam Yaiphaba Singh	**(2201CS0102)**
Dayananda Laishram	**(2201CS0113)**

Under the Guidance of

Dr. Roshni Rajkumari
Assistant Professor

Department of Computer Science & Engineering

Manipur Technical University

May, 2026

DEDICATION

This thesis is dedicated to all those who have inspired, supported and stood by us throughout the journey of this project.

To our families: Your unwavering love, patience, and encouragement have been the bedrock of our emotional strength during the most challenging phases. *Your belief in our potential has fortified our resilience and determination.*

To our mentors and instructors: Your expertise, thoughtful guidance, and commitment to our academic growth have been instrumental in shaping both this project and our understanding of the field. *Your encouragement and constructive feedback have been invaluable at each stage.*

To our friends and peers: Your camaraderie, moral support, and motivation have illuminated our path. *Your presence transformed moments of stress into opportunities for shared growth and learning.*

To the global community of researchers, engineers, and innovators in Artificial Intelligence and Computer Vision: Your pioneering work has laid the foundation for this thesis. *Your dedication to advancing knowledge continues to inspire and propel us to contribute meaningfully to this dynamic field.*

This thesis stands as a testament to the collective support, shared vision, and unwavering encouragement of all who have been part of our academic and personal journey.

AI-Based Paddy Seed Quality Assessment And Farmer Feedback System

A thesis submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

By

Joymangol Chingangbam	**(2201CS0004)**
Nilambar Elangbam	**(2201CS0005)**
Khangembam Yaiphaba Singh	**(2201CS0102)**
Dayananda Laishram	**(2201CS0113)**

Under the Guidance of

Dr. Roshni Rajkumari
Assistant Professor

Department of Computer Science & Engineering

Manipur Technical University

May, 2026

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

BONAFIDE CERTIFICATE

This is to certify that the thesis entitled **"AI-Based Paddy Seed Quality Assessment and Farmer Feedback System"** submitted by:

Joymangol Chingangbam	(2201CS0004)
Nilambar Elangbam	(2201CS0005)
Khangembam Yaiphaba Singh	(2201CS0102)
Dayananda Laishram	(2201CS0113)

to the **Department of Computer Science & Engineering**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree in **Computer Science and Engineering**, is a genuine and original work carried out by them under my supervision during the academic year **2025–2026**.

The project embodies the results of their own research and development work and has not been submitted, either in part or in full, for the award of any other degree or diploma of this university or any other institution. The work has been carried out with sincerity, academic integrity, and adherence to research ethics.

We consider this work worthy of consideration for the partial fulfillment of the requirements for the said degree.

Supervisor

Dr. Roshni Rajkumari
Assistant Professor,
Computer Science & Engineering

Head of Department

Mrs. Tayenjam Aerena
Assistant Professor & Head,
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I hereby declare that the thesis entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"**, submitted to the **Department of Computer Science and Engineering, Manipur Technical University**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree, is a result of my own work and effort. Under the guidance of **Dr. Roshni Rajkumari**, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University.

I affirm that this work has been carried out with academic honesty and integrity. All sources of information have been properly cited and acknowledged. This thesis has not been submitted earlier, either in part or in full, for the award of any degree or diploma at this or any other institution.

Date:
Place:

Joymangol Chingangbam
Reg. No. - 2201CS0004
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I hereby declare that the thesis entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"**, submitted to the **Department of Computer Science and Engineering, Manipur Technical University**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree, is a result of my own work and effort. Under the guidance of **Dr. Roshni Rajkumari**, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University.

I affirm that this work has been carried out with academic honesty and integrity. All sources of information have been properly cited and acknowledged. This thesis has not been submitted earlier, either in part or in full, for the award of any degree or diploma at this or any other institution.

Date:
Place:

Nilambar Elangbam
Reg. No. - 2201CS0005
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I hereby declare that the thesis entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"**, submitted to the **Department of Computer Science and Engineering, Manipur Technical University**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree, is a result of my own work and effort. Under the guidance of **Dr. Roshni Rajkumari**, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University.

I affirm that this work has been carried out with academic honesty and integrity. All sources of information have been properly cited and acknowledged. This thesis has not been submitted earlier, either in part or in full, for the award of any degree or diploma at this or any other institution.

Date:
Place:

Khangembam Yaiphaba Singh
Reg. No. - 2201CS0102
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I hereby declare that the thesis entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"**, submitted to the **Department of Computer Science and Engineering, Manipur Technical University**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree, is a result of my own work and effort. Under the guidance of **Dr. Roshni Rajkumari**, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University.

I affirm that this work has been carried out with academic honesty and integrity. All sources of information have been properly cited and acknowledged. This thesis has not been submitted earlier, either in part or in full, for the award of any degree or diploma at this or any other institution.

Date:
Place:

Dayananda Laishram
Reg. No. - 2201CS0113
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

ACKNOWLEDGEMENT

We sincerely acknowledge the collective support and guidance received throughout the successful completion of our thesis project **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System".**

We take this opportunity to thank our Vice Chancellor, **Prof. W. Chandbabu Singh**, whose aid and encouragement provided invaluable support and inspiration throughout our academic journey. We are deeply grateful to the Department of Computer Science and Engineering, Manipur Technical University, for providing the academic environment, infrastructure and encouragement that fostered innovation and technical exploration.

Our heartfelt thanks go to our project supervisor, **Dr. Roshni Rajkumari**, Assistant Professor, whose consistent guidance, insightful feedback and unwavering support played a pivotal role in shaping this project. We also extend our sincere appreciation to **Mrs. Tayenjam Aerena**, Head of Department, for her inspiring leadership and motivation.

We are also thankful to all the faculty members, technical staff and friends who provided their valuable input and moral support throughout the journey.

Date:
Place:

Sincerely,
Joymangol Chingangbam
Nilambar Elangbam
Dayananda Laishram
Khangembam Yaiphaba Singh

ABSTRACT

The quality of paddy seed plays a vital role in crop yield, germination rate and overall agricultural productivity. Many small and marginal farmers suffer crop loss due to substandard or counterfeit seeds that closely resembles certified products. Due to lack of affordable and rapid verification methods, farmers rely on packaging labels and basic manual inspection, which is time consuming, inconsistent and error prone. Poor quality seeds are generally known after the seeds are sown, and thus, results in low germination, reduce yield and incur a financial loss.

To address these challenges, our project entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"** is designed to combat the spread of counterfeit and substandard paddy seeds. This project is a web-based platform that utilizes artificial intelligence and deep learning model to automate the classification of quality seeds into healthy and suspicious ones based on characteristics like size, shape uniformity, colour, surface texture and visible defects. Farmers can scan seed samples using a simple image-based interface to receive an estimated quality assessment of paddy seeds, presented in clear, farmer friendly language.

In addition to seed quality assessment, our system also includes a Farmer Complaint and Feedback module that enables reporting of field level issues such as poor germination or crop failure, cultivation experience, seed performance, etc.

The system avoids chemical analysis, specialized hardware or complex supply chain systems, making it low-cost, scalable and suitable for rural deployment. It also helps farmers select better quality seeds, reduce crop failure, and improve productivity. The integration of artificial intelligence with farmer feedback system makes the proposed system a useful tool for modern agricultural practices and sustainable farming development.

Table of Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 Background and Context

Agriculture is the foundational backbone of many global economies, heavily influencing food security, employment and national income. At the core of all agricultural productivity depends on the quality of the seed planted, which is the most critical and non-negotiable factor determining crop yield, resilience to disease and overall harvest quality. However, the agricultural sector is currently facing a severe crisis due to the increasing problem of counterfeit or fake seeds in the market.

Counterfeit seeds are often packaged to resemble premium brands or are visually indistinguishable from high-yield varieties, yet they exhibit drastically lower germination rates and poor genetic traits. Traditional methods of verifying seed authenticity rely on destructive laboratory testing or subjective physical inspection, both of which are inaccessible, expensive and far too slow for the an average farmer. Consequently, farmers often discover that they have been deceived only after planting, leading to catastrophic losses in time, resources and income.

1.2 Motivation

The primary motivation for this project stems from the devastating socio-economic impact that low-quality seeds have on farming communities. When farmers unknowingly purchase and plant low-quality seeds, they face severe financial distress, which cascades into broader issues of local food insecurity and loss of trust in agricultural supply chains. There is an urgent, unmet need for accessible technological tools that empower grassroots agricultural workers to protect

themselves against fraud.

Simultaneously, the rapid advancement of deep learning, computer vision, and cloud computing presents an unprecedented opportunity to address this crisis. By leveraging deep learning and computer vision techniques it is now possible to achieve automated, laboratory-grade visual analysis of seeds. The motivation is to harness these cutting-edge AI technologies and package them into a simple, web-based tool, putting the power of automated, real-time seed verification directly into the hands of farmers.

1.3 Problem Statement

Despite the devastating effects of agricultural fraud, farmers currently lack a reliable, accessible, and rapid method to accurately distinguish between genuine and counterfeit seeds before the planting season begins. Visual similarities make manual detection nearly impossible and existing institutional supply chains lack a transparent, real-time verification system.

The core problem is the absence of an integrated, technology-driven solution that can automatically analyze seed imagery, identify fraudulent anomalies and capture feedback from farmers to map and identify the spread of counterfeit seeds in real-time.

1.4 Objectives

The main goal of this project is to create an intelligent ecosystem for seed verification. To achieve this, the project outlines the following specific objectives:

1. To design and train a deep learning model capable of accurately classifying seed images and identifying anomalies associated with counterfeit seeds.
2. To develop and deploy a user-friendly web-based platform that allows farmers to upload images of their seeds for AI inference and verification.
3. To integrate a "Farmer Feedback System" within the platform, enabling users to report their post-purchase and post-planting experiences (such as germination failure rates).
4. To utilize the feedback and newly uploaded images to continuously refine the database, creating an early warning system against emerging counterfeit supply chains.

1.5 Scope of the Project

The scope of this project encompasses the end-to-end development of an AI-driven web application tailored for agricultural quality control. This includes the collection, curation, and preprocessing of a targeted seed image dataset, followed by the training, validation, and optimization of a Convolutional Neural Network (CNN).

Additionally, the scope covers the full-stack development of the web platform, including an intuitive frontend interface for image uploads and a robust backend to handle model inference and database management. The project is strictly limited to the non-destructive, visual assessment of seeds via digital imagery; it does not extend to biochemical, genetic, or DNA-based testing methods. The final deliverable will be a functional prototype that demonstrates the feasibility of combining deep learning with crowdsourced feedback to combat agricultural fraud.

Chapter 2

Literature Review

2.1 Seed Quality Assessment: Traditional Methods

Conventional seed quality assessment rests on two broad approaches: manual visual inspection and laboratory-based biochemical testing. In field practice, inspectors evaluate seeds by examining size, shape, colour and surface texture; judgements that are inherently subjective, vary between observers and resist standardisation across districts or seasons. Laboratory tests such as the Tetrazolium (TZ) test and standard germination assays offer more reliable viability estimates, but both are destructive and typically require four to seven days under controlled conditions. That turnaround makes them impractical for screening large consignments at intake points, or for farmers who need a decision before the planting window closes.

Near-infrared (NIR) spectroscopy and X-ray transmission imaging have been investigated as non-destructive alternatives. NIR detects internal moisture gradients and biochemical composition without damaging the sample; X-ray imaging can reveal embryo defects invisible on the seed surface. Both perform well in laboratory conditions, but equipment costs typically USD 15,000-80,000 per unit and the requirement for trained operators confine their use to national seed-testing stations. Smallholder farmers and district-level inspectors in regions such as northeast India have no practical access to either technology, which is precisely where the need for affordable, rapid screening is most acute.

2.2 Deep Learning for Agricultural Image Classification

Convolutional Neural Networks (CNNs) have been applied to a range of agricultural classification problems over the past decade, including leaf disease iden-

tification [1], weed recognition [2], and post-harvest quality grading [3]. Their principal advantage over classical machine-learning pipelines is that spatial features are learned directly from raw pixel data during training, removing the need to manually engineer feature extractors for each new crop or defect type.

Applying CNNs to seed classification follows naturally from this body of work. Paddy seeds exhibit subtle differences in surface texture and colour between certified and counterfeit lots; differences that resist quantification through handcrafted descriptors but emerge reliably from sufficiently large training sets. The computational cost of running a trained CNN at inference time is also modest enough to support server-side web deployment, which matters when target users have intermittent electricity but reasonable mobile-data coverage.

2.3 Transfer Learning and ResNet Architectures

Training a CNN from random initialisation requires on the order of tens of thousands of labelled examples per class to achieve stable convergence. Annotated seed image datasets of that scale do not yet exist for most crop varieties, and certainly not for the specific task of detecting counterfeits. Transfer learning is the standard response: a model pre-trained on ImageNet (1.28 million labelled images across 1,000 categories) is fine-tuned on the smaller domain-specific dataset. The early convolutional layers, which encode low-level features such as edges and textures, are retained; only the later task-specific layers are updated. In practice, this approach consistently reaches accuracy close to what a from-scratch model would achieve on a dataset roughly ten times larger.

ResNet architectures are well-suited to this fine-tuning scenario. The residual skip connections introduced by He et al. [4] allow gradients to propagate through very deep networks without vanishing, making it feasible to fine-tune models with 50 or more layers on datasets that would cause shallower architectures to overfit. ResNet-50 has become a common baseline in agricultural image classification, balancing parameter count ($\approx$25 million) against accuracy on standard benchmarks, with pre-trained weights publicly available. For this project, it provides a well-validated starting point that reduces both training time and the risk of overfitting on the available paddy seed dataset.

2.4 Farmer Feedback Systems

A detection model trained on a fixed dataset will degrade over time if counterfeiting practices evolve and no new training data are collected. Field-level feedback from farmers is one practical mechanism for capturing these shifts. When a farmer submits a seed batch for image-based verification and subsequently reports germination failure or abnormal yield, that report constitutes a labelled example, a model prediction paired with a ground-truth outcome, that can be incorporated into periodic retraining cycles.

Feedback also serves a more immediate diagnostic function. Aggregating reports across users can flag a suspect lot or supplier before the model itself is updated: three independent failure reports from the same district within a fortnight is a meaningful signal, regardless of what the image classifier predicts. This rule-based alerting complements the model rather than replacing it. Sustaining farmer participation remains a practical difficulty; deployments in low-resource contexts consistently find that single-tap rating interfaces produce far higher response rates than free-text fields or structured forms [5].

2.5 Web-Based Agricultural Platforms

Deploying a trained model as a web service separates the computational workload from the end user's device. A farmer uploads a seed image through a browser; the server runs inference and returns a result. This architecture offers two concrete advantages over distributing a standalone mobile application: the model can be updated centrally without requiring client-side installations, and every prediction is logged in a structured format that supports monitoring and retraining.

For farmers in Manipur and comparable regions, browser-based access via a mid-range Android handset is more realistic than assuming a dedicated application installation. Keeping the frontend lightweight, avoiding heavy JavaScript bundles and large media assets reduces page-load times on 4G connections with variable signal quality. Containerised inference endpoints handle computationally intensive tasks without requiring a permanently running server, reducing operating costs during off-season periods when query volume is low.

2.6 Review of Related Work

Early automated seed classification employed Support Vector Machines (SVMs) and Random Forests operating on handcrafted features derived from shape measurements, colour histograms, and texture descriptors. Ni et al. [6] classified rice seed varieties using morphological parameters extracted from binary silhouettes, achieving 94% accuracy across five varieties under controlled lighting. These methods perform adequately when intra-class variation is low and imaging conditions are uniform, but generalise poorly to photographs taken in field settings.

Deep learning approaches have largely displaced handcrafted methods in more recent work. Liu et al. [7] applied VGG16 transfer learning to maize seed defect detection, reporting 97.3% accuracy on a 12-class dataset. Xu et al. [8] used ResNet-50 for soybean quality grading and found that fine-tuning from ImageNet weights outperformed training from scratch by approximately 8 percentage points when training data were limited to fewer than 2,000 images per class. Both studies, however, address naturally occurring defects, such as cracking, mould and insect damage, rather than intentional substitution. A counterfeit seed is engineered to resemble the genuine article; distinguishing the two is a harder classification problem than separating a damaged kernel from an intact one. The visual gap between classes is narrower, and the adversarial nature of counterfeiting means the distribution of counterfeits may shift as producers adapt. No published study directly evaluates CNN performance on verified counterfeit-versus-genuine seed pairs, and none integrates a structured feedback collection mechanism into the evaluation pipeline.

2.7 Summary and Research Gaps

The reviewed literature establishes that transfer-learned CNNs are effective for agricultural image classification, including seed quality assessment. Three gaps are directly relevant to this work. First, no published study has targeted intentional counterfeiting as distinct from natural quality defects; the two problems differ in that counterfeits may pass casual visual inspection and require the model to detect subtle discriminative cues. Second, existing models are evaluated on laboratory-quality images collected under controlled conditions; performance on photographs taken by farmers using mobile devices in variable lighting has not been reported. Third, no end-to-end system has been published that connects model inference to a structured feedback collection mechanism and uses that feedback iteratively for retraining.

This project addresses all three gaps. The classifier is trained and evaluated specifically on a fake-versus-genuine paddy seed dataset; the web platform is tested with mobile-captured images; and the farmer feedback module is built directly into the application, with logged responses stored in a format suitable for future retraining rounds.

Chapter 3

System Design and Architecture

3.1 Overall System Architecture

Building AgriVerify required us to carefully orchestrate three key layers: a deep learning system for analyzing seed images, a robust backend API to manage the workflow, and a user-friendly interface for farmers and officials. We chose the **Clean Architecture** approach (specifically the Onion Architecture variant) because it keeps the core business logic completely separate from external concerns like databases and cloud platforms. Put simply, dependencies always point inward—the outer layers depend on the inner ones, never the other way around [9].

At the heart of our system are three main components working together:

1. **Deep Learning Microservice** — We trained a PyTorch-based ResNet-50 model on 1,563 paddy seed images and deployed it as a containerised FastAPI service on Google Cloud Platform's Cloud Run. This service handles the heavy lifting of image analysis [4].

2. **Backend API** — We built an ASP.NET Core 8 Web API that acts as the central hub. It manages user authentication, communicates with the ML model and various Azure AI tools (for image classification, text extraction, and intelligent report generation), stores results in a SQL Server database, and provides a clean REST interface for the frontend to consume [10].

3. **Frontend Web Application** — A Next.js 14 application that delivers different dashboards for farmers, agricultural officers, and system administrators. To keep things secure, all communication between the browser and backend is routed through a server-side proxy, ensuring authentication tokens never leak to the client side [11].

3.1.1 Backend Architectural Layers

Our backend follows a four-layer design pattern, with each layer handling a specific set of responsibilities. The core responsibilities and components encapsulated within each layer are detailed in Table 3.1.

Table 3.1: Backend Architectural Layers and Responsibilities

Architectural Layer	**Core Responsibilities and Encapsulated Components**
Domain Layer	This is where we define the heart of our application: core entities like `User` and `VerificationHistory`, value objects, and error types. Importantly, this layer has zero dependencies on external frameworks or services—it's pure business logic.
Application Layer	Think of this as the conductor of our application. It orchestrates how different business operations flow together using application services (`VerificationService`, `UserService`), data transfer objects (DTOs) for communication, mapping configurations, and validation rules. It depends only on the Domain layer.
Infrastructure Layer	This layer bridges our application to the outside world. It provides concrete implementations of interfaces, manages database interactions via Entity Framework Core, wraps Azure services (`CustomVisionService`, `ComputerVisionService`, `OpenAIService`), and handles file storage. The details here stay hidden from the upper layers.
API Layer	This is the front door of our system. It hosts the ASP.NET Core controllers that handle HTTP requests, manages JWT authentication, exposes API documentation via Swagger, and handles errors consistently across all endpoints.

3.2 Functional Requirements

We spent considerable time talking to farmers, agricultural officers, and domain experts to understand exactly what AgriVerify needed to do. These conversations shaped the following requirements, which became our roadmap for what to build.

Table 3.2: Comprehensive Functional Requirements Specification

Req ID	Module / Role	Functional Specification
FR-01.1	**Farmers (Auth)**	Farmers can sign up using their name, email, state, and district information. They log in with their email and password.
FR-01.2	**Officers (Auth)**	Agricultural officers authenticate using a unique officer ID and a time-based one-time password (TOTP) for added security.
FR-01.3	**Administrators**	Admin accounts are set up through initial database setup; their credentials are managed directly by the system.
FR-01.4	**Session Control**	When users log in successfully, the system issues them access tokens and refresh tokens. These tokens become invalid when users log out.
FR-02.1	**Image Submission**	Farmers can upload photos of the front and back of their seed packages so the system can analyze them.
FR-02.2	**AI Inference**	The system simultaneously uses three AI tools: one to identify the packaging genuineness, one to read text from the packaging, and one to extract physical seed characteristics.

Continued on next page

Table 3.2 continued from previous page

Req ID	Module / Role	Functional Specification
FR-02.3	**Result Output**	After analysis, the system provides a confidence score, a clear verdict (Genuine / Counterfeit / Suspicious), the extracted text from the packaging, and a summary report written in plain language.
FR-02.4	**Public Access**	Anyone—even without creating an account—can use the verification system to check if their seeds are genuine.
FR-03.1	**Farmers (Complaints)**	When farmers encounter a problem with seeds, they can file a complaint describing the issue, which district it affects, how serious it is, and can attach photos if needed.
FR-03.2	**Officers (Complaints)**	Officers assigned to a complaint can view its details, update its status as they investigate, and document what actions they've taken and when.
FR-03.3	**Admin (Complaints)**	Administrators can see all complaints across the system, filter them by location and status, and assign unhandled complaints to available officers.
FR-04.1	**Farmer Dashboard**	Farmers have a personal dashboard showing their past verifications, the status of any complaints they filed, and quick summary numbers.
FR-04.2	**Officer Dashboard**	Officers see a prioritised list of complaints, can quickly assess risk across batches of seeds, and have access to helpful analytics.

Continued on next page

Table 3.2 continued from previous page

Req ID	Module / Role	Functional Specification
FR-04.3	**Admin Dashboard**	Admins have a complete view of system performance across all users and locations, can manage user accounts, and can adjust reference data as needed.
FR-05.1	**Data Aggregation**	We maintain a registry (`BatchRiskRegistry`) that collects complaint data by batch number, making it easy to spot problematic seed batches at a glance.
FR-05.2	**Risk Analytics**	The system tracks which districts are affected, calculates average severity scores, and labels each batch with a risk category (Low, Medium, High).
FR-06.1	**Feature Inference**	When a seed image arrives at our ML microservice, it preprocesses the image, runs it through the ResNet-50 model, and extracts 17 physical measurements like seed length, width, color, and texture.
FR-06.2	**Variety Matching**	The microservice compares the seed against our database of 12 registered paddy varieties and returns the three best matches along with similarity scores.

3.3 Non-Functional Requirements

Beyond what the system does, we also cared deeply about how well it does it. These non-functional requirements shaped our architectural decisions at every turn.

Table 3.3: Comprehensive Non-Functional Requirements Specification

Req ID	Quality Attribute	Architectural Constraint & Target Specification
NFR-01	**Performance**	When a farmer uploads seed images, we want an answer back quickly—ideally within a few seconds. To achieve this, we run image analysis and text extraction simultaneously rather than one after the other, and our ML model is optimized to deliver results in well under a second.
NFR-02	**Scalability**	Our backend doesn't keep any state tied to individual servers, so we can easily run multiple copies of it to handle more users. The frontend uses server-side rendering and edge caching to handle traffic spikes gracefully, especially on mobile networks.
NFR-03	**Security**	All communication happens over secure HTTPS connections. Access tokens are short-lived and minimal. Passwords are securely hashed using industry-standard algorithms. Sensitive credentials like API keys and database passwords are kept in secure environment variables, never in code. Role-based access control ensures users can only do what they're authorized to do [12].
NFR-04	**Reliability & Data Integrity**	Database operations that involve multiple changes happen atomically—either all succeed or all fail, preserving data integrity. We use soft deletes so complaint and verification records can be preserved for future audits rather than permanently removed. The system includes health checks to monitor whether critical services are working properly.

Continued on next page

Table 3.3 continued from previous page

Req ID	Quality Attribute	Architectural Constraint & Target Specification
NFR-05	**Maintainability**	We've organized our code using Clean Architecture so it's easy for future developers to understand and modify. Infrastructure setup is handled cleanly through dedicated helper classes rather than scattered throughout startup code. All API endpoints are fully documented so other developers know how to use them.
NFR-06	**Usability & Accessibility**	The interface works smoothly on phones and desktop computers, since many farmers access AgriVerify via smartphones. Users get immediate visual feedback—loading indicators, success messages—so they know the system is working.
NFR-07	**Portability**	We've exported our trained model in multiple formats so it can run in different environments and on different platforms. The entire system can be containerized and deployed anywhere that supports Docker.

3.4 Tools and Technologies

3.4.1 Deep Learning Stack (PyTorch and ResNet-50)

We built our seed classification model in **Python 3.10** using these core libraries [13]:

- **PyTorch** (`torch`, `torchvision`) — This is our primary framework for building and training the neural network. We chose it because its dynamic architecture made debugging easier and allowed us to experiment with different model designs during development.
- **ResNet-50** — We started with a 50-layer deep neural network that was

pre-trained on 1.2 million images across 1,000 different object categories. This pre-training gave us a head start; we then fine-tuned it specifically for seed classification using `torchvision.models` [4]. The base model contains 23.5M parameters; we added a custom classification head with another 1.1M trainable parameters tailored to our three output classes.

- **OpenCV** (`cv2`) — We used this library to process seed images and extract 17 physical measurements. The process involves finding seed boundaries, measuring pixel dimensions relative to a known reference, and analyzing seed shape characteristics like how round or solid the seed is [14].
- **NumPy / Pandas** — We used these for handling numerical arrays and organizing our dataset statistics [15].
- **scikit-learn** — This provided handy utilities for splitting data into training, validation, and test sets while maintaining data balance, plus functions for calculating evaluation metrics [16].
- **Matplotlib / Seaborn** — We created visualizations to understand how well the model learned and to spot any patterns in its errors.

We trained the model on a GPU using AdamW (an optimizer that also regularizes weights with $\lambda = 10^{-4}$), cross-entropy loss with label smoothing ($\varepsilon = 0.1$), and a two-stage training approach. In the first stage (epochs 1–8, learning rate $= 10^{-3}$), we froze the pre-trained backbone and only trained the new classification head so it could quickly learn what features matter for seed classification. In the second stage (epochs 9–15, learning rate $= 10^{-4}$), we unfroze the entire network and fine-tuned every layer end-to-end. The final model achieved **92.15%** accuracy on our test set of 1,563 seed images, with the training accuracy only about 2.8% higher than validation accuracy—indicating the model generalizes well to new images.

3.4.2 Backend Stack (ASP.NET Core 8, SQL Server, and Azure)

We built the backend REST API using the **Microsoft .NET 8** ecosystem. A comprehensive summary of the core technologies, libraries, and their designated functions within the backend stack is provided in Table 3.4:

We wrapped each Azure AI service in a dedicated class (`CustomVisionService`, `ComputerVisionService`, `OpenAIService`) that handles the details of communicating with Azure. These are registered once at startup and reused

Table 3.4: Backend Technology Stack

Component	**Technology / Library**
Web Framework	ASP.NET Core 8.0 / .NET 8 (Kestrel)
Database	Microsoft SQL Server
ORM	Entity Framework Core (Code-First, Migrations)
Authentication	ASP.NET Core Identity + JWT Bearer
Object Mapping	AutoMapper
Validation	FluentValidation + DataAnnotations
Logging	Serilog (Console and File sinks)
API Documentation	Swagger / OpenAPI 3.0
AI Services	Azure Custom Vision, Azure AI Vision (OCR), Azure OpenAI
Cloud Storage	Azure Blob Storage
Deployment	Docker / Render Cloud

throughout the application. The clever part is using parallel execution (`await Task.WhenAll`) so that when a farmer submits images, we analyze the packaging, extract text, and run the physical analysis all at the same time—cutting down the total wait time [10].

3.4.3 Model Serving (FastAPI and Docker)

Once we trained our PyTorch model, we wrapped it in a **FastAPI** microservice and deployed it as a Docker container on **GCP Cloud Run** so it can automatically scale based on demand [17]. Here's how the inference pipeline works step by step:

1. **Request** — The backend sends a seed image (as multipart file data) to the FastAPI endpoint.

2. **Preprocessing** — The image is resized to 224×224 pixels and normalized using standard ImageNet values $\boldsymbol{\mu} = [0.485, 0.456, 0.406]$ and $\boldsymbol{\sigma} = [0.229, 0.224, 0.225]$ so the model sees consistent inputs.

3. **Model Inference** — ResNet-50 processes the normalized image and produces raw classification scores (logits).

4. **Post-processing** — We convert those scores to probabilities using softmax, extract the 17 physical seed measurements using OpenCV, and compare the seed against known varieties in our reference database.

5. **Response** — A JSON payload is sent back containing the predicted class, confidence score, physical measurements, and the top-3 matching varieties with their similarity percentages.

To maximize deployment flexibility, we export the model in three formats: native PyTorch (`.pth`, ≈98 MB), TorchScript for C++ environments (`.pt`, ≈97 MB), and ONNX for cross-platform inference (`.onnx`, ≈98 MB). We've set up automated deployment using GitHub Actions—whenever we push new code to the main branch, the system automatically builds a Docker image and deploys it to GCP.

3.4.4 Frontend Stack (Next.js, TypeScript, and TanStack Query)

The user-facing interface is built with **Next.js 14 App Router** and **TypeScript**. Here's what makes it tick:

- **Next.js 14 App Router** — This framework handles routing and rendering. We organized our routes into three separate sections (`(farmer)`, `(officer)`, `(admin)`) so each user type sees their own personalized interface. Some pages render on the server for better performance; interactive elements render in the browser.
- **TypeScript** — This adds type checking at development time, catching errors before code runs. We define shared data structures (`src/types/`), service modules (`src/services/`), and custom hooks (`src/hooks/`) all with full type safety.
- **TanStack Query (React Query v5)** — Manages all the complex state around fetching data from the server: caching, refreshing, pagination, and rolling back failed changes. Query functions live alongside their domain logic; we've also configured it to not automatically refetch when the browser tab comes back into focus, reducing unnecessary requests [18].
- **Zustand** — A lightweight state store for authentication info (`user`, `token`, `role`, `isAuthenticated`). On page load, it reads from the browser's local storage; when users log out, it clears everything [19].
- **Axios** — Our HTTP client configured to talk to the backend via `/api/proxy`. It automatically adds authentication tokens to requests and handles the scenario where a token expires mid-request by getting a fresh token and retrying [20].

- **Next.js API Route Proxy** — For security, we added a server-side proxy at `/api/proxy/[...path]` that forwards requests from the browser to the actual backend API. This way, authentication tokens stay on the server and never appear in browser JavaScript.
- **Tailwind CSS** — A utility-first CSS framework that lets us quickly style components. We extended the default design with custom colors, border styles, and shadows in `tailwind.config.ts`.
- **Shadcn/UI + Custom Primitives** — We built a compact component library with frequently-used UI elements (buttons, input fields, dialogs, tables, badges) built on standard HTML and styled with Tailwind, keeping things lightweight.

3.5 System Architecture

3.5.1 High-Level Architecture

Figure 3.1 shows how the whole system fits together. When a farmer or officer uses AgriVerify through their browser, they never directly connect to the backend—everything goes through the Next.js frontend. The frontend acts as a trusted intermediary: it injects the secure authentication token into requests and forwards them to the ASP.NET backend. The backend then talks to three specialized services: the ML microservice on GCP for image analysis, Azure AI tools for text extraction and packaging verification, and finally stores everything in a SQL Server database. Any seed images are also safely stored in Azure's cloud storage.

3.5.2 Data Flow Diagram

Figure 3.2 walks through what happens when a farmer submits seed images for verification. The request travels from the browser through the Next.js security layer into the ASP.NET backend, where the system springs into action: it simultaneously asks the ML microservice to analyze the seed image, sends the image to Azure for packaging authenticity checks, and extracts any text visible on the package. Once all three analyses finish, Azure's AI synthesizes the results into a clear, easy-to-understand report. This report is saved to the database and sent back to the farmer's screen.

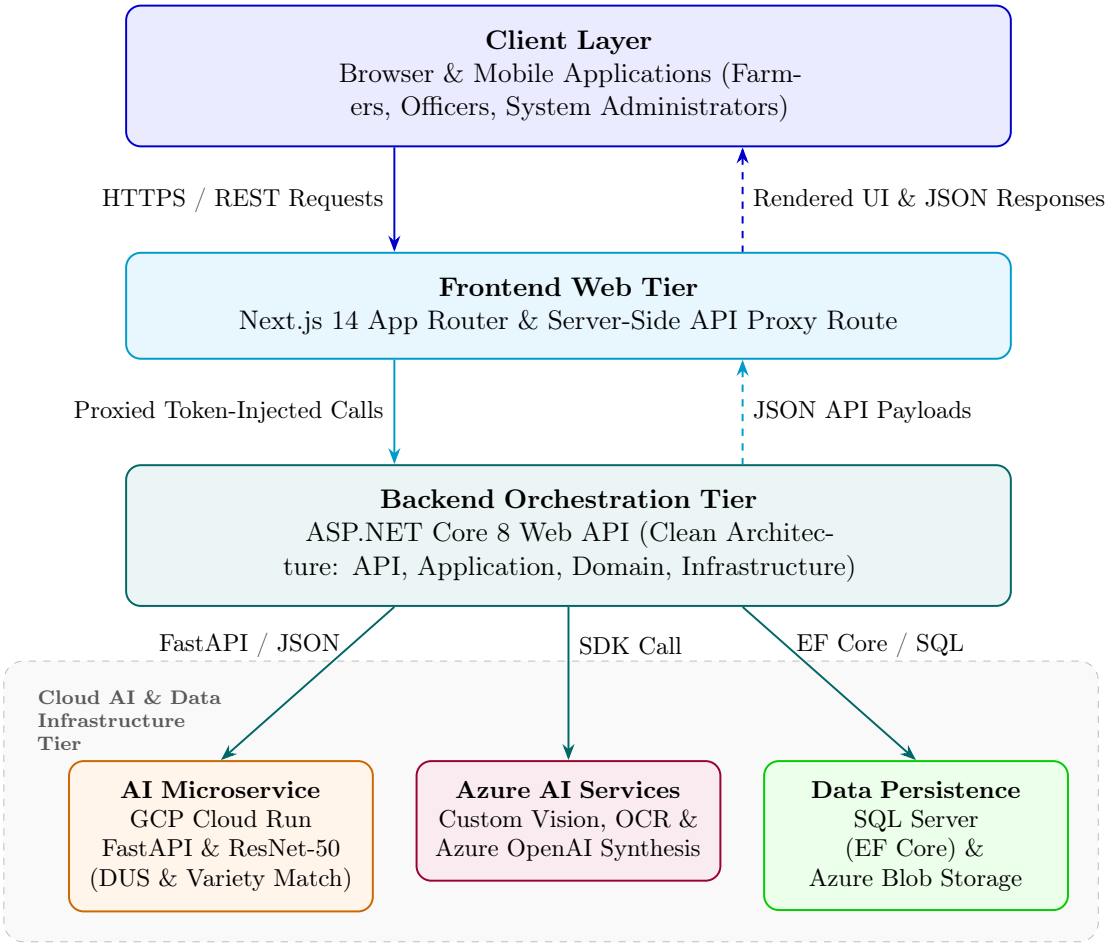


Figure 3.1: High-level system architecture of the AgriVerify platform.

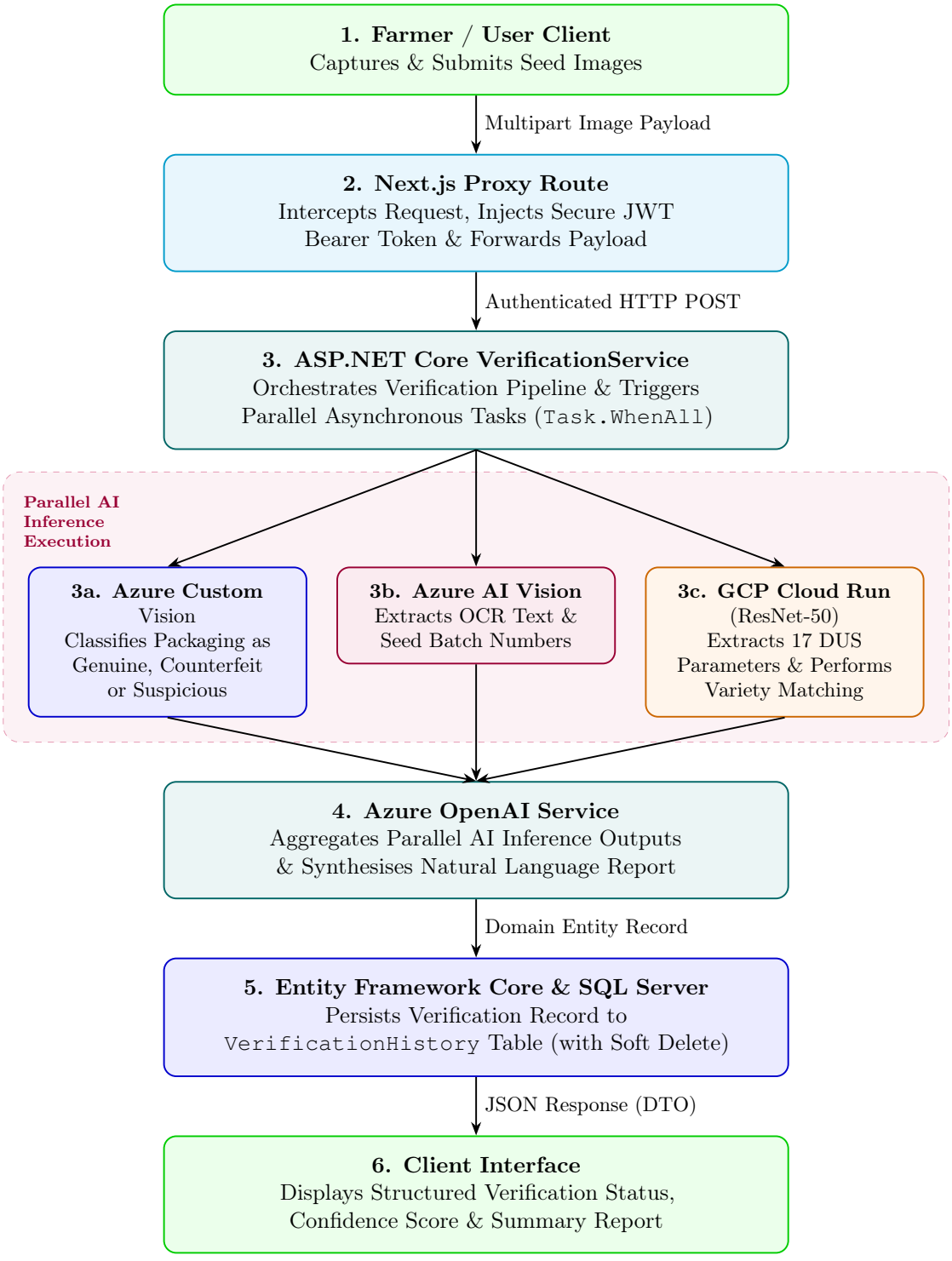


Figure 3.2: Data flow diagram for the seed verification pipeline

3.5.3 Entity-Relationship Diagram

Figure 3.3 shows the main database tables and how they connect to each other. Think of this as the backbone of AgriVerify's data storage—it captures users, their verification history, complaints filed against problematic seeds, and the actions officers take to investigate those complaints.

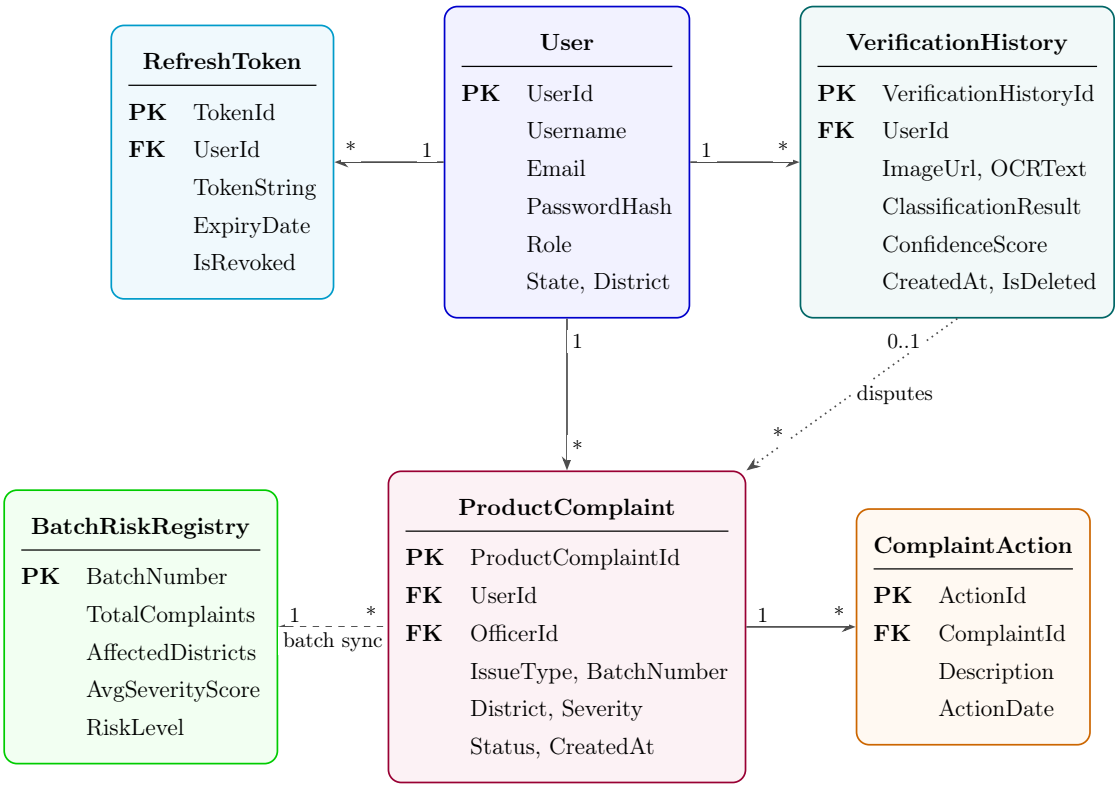


Figure 3.3: Simplified entity-relationship diagram for the AgriVerify domain model. Cardinality notation follows UML conventions.

A few key design choices in this schema are worth mentioning:

- **Soft delete** — Rather than permanently removing verification records or complaints from the database, we mark them as deleted. This way, we preserve a complete audit trail that might be needed for legal investigations or disputes down the road.
- **Smart database design** — We split seed data, verification results, and complaint information into separate tables. This eliminates redundancy and makes sure that when we update information in one place, it's consistent everywhere it appears.
- **Fast lookups** — We've added database indices on frequently-queried columns like email addresses, user IDs associated with verifications, and complaint status filters. This keeps dashboard queries snappy even as the system grows.

- **BatchRiskRegistry** — This table pre-computes risk summaries for each seed batch whenever a new complaint comes in. Rather than recalculating these statistics every time an officer checks the dashboard, we just read from this table. It's a smart tradeoff between performance and simplicity.

Chapter 4

Deep Learning Model for Paddy Seed Quality Assessment

4.1 Dataset Description

4.1.1 Data Sources and Collection (Kaggle)

We built our training dataset by combining two sources from Kaggle. The first is a publicly available dataset, and the second is a custom collection we put together in partnership with India's Council of Agricultural Research (ICAR) facility in Manipur. By bringing these two sources together, we got a larger and more realistic dataset—one that includes seed images captured in different ways and from different parts of India. This diversity is crucial because it helps the model learn to recognize seeds regardless of lighting conditions or camera setup.

4.1.1.1 Public Source

The first source is the Paddy Seeds Quality Classification dataset published on Kaggle by Ayyan Arkadalkani:

`https://www.kaggle.com/datasets/ayyanarkadalkani/paddy-seeds-quality-classification-dataset`

This dataset contains 1,213 labeled images split into three categories: Healthy, Damaged, and Fake/Low Quality seeds. The images take up about 183 MB of storage. These images were captured under many different lighting conditions and backgrounds, which made them a solid foundation for our model to learn general patterns about what different seed qualities look like.

4.1.1.2 Custom Source

The second source is a dataset we collected directly from ICAR's Manipur laboratory. ICAR researchers took photographs of paddy seeds under controlled lab conditions, focusing on the specific varieties grown in northeastern India. This custom collection includes 350 images occupying about 24.7 MB of storage. We've published this dataset on Kaggle as an open resource so other researchers can use it:

`https://www.kaggle.com/datasets/nilambarelangbam/paddy-seed-images-collected-from-icar-manipur`

When we combined both datasets, we ended up with 1,563 images that represent a wider variety of seed conditions than either dataset alone. This combined dataset better reflects what farmers and agricultural officers actually encounter in the field across different regions of India.

4.1.2 Dataset Statistics and Class Distribution

After merging the two datasets, we had a total of 1,563 paddy seed images divided into three quality categories. A detailed breakdown of the datasets, their image counts, storage sizes, and public Kaggle repositories is summarized in Table 4.1, while the class-wise distribution of these images is provided in Table 4.2.

Table 4.1: Dataset Source Summary

Source	**Images**	**Size**	**Kaggle Identifier**
Paddy Seeds Quality Classification (Ayyan Arkadalkani)	1,213	∼183 MB	`ayyanarkadalkani/paddy-seeds-quality-classification-dataset`
ICAR Manipur Custom Dataset (Our Collection)	350	∼23.7 MB	`nilambarelangbam/paddy-seed-images-collected-from-icar-manipur`
Combined Total	**1,563**	**∼208 MB**	—

As you can see, we have more healthy seed images than damaged or fake ones—which makes sense because in real life, most seeds are healthy. However, this imbalance could cause the model to become biased toward recognizing healthy seeds and miss the problematic ones. To fix this, we used a technique called weighted sampling during training: when building each training batch, we ensured each

Table 4.2: Class-wise Distribution of the Combined Dataset

Class	**Number of Images**	**Percentage (%)**
Healthy Paddy Seeds	∼750	∼48.0
Damaged Paddy Seeds	∼510	∼32.6
Fake/Low Quality Seeds	∼303	∼19.4
Total	**1,563**	**100.0**

quality category contributed fairly, regardless of how many examples we had of each type.

4.1.3 Train/Validation/Test Split Strategy

We divided the 1,563 images into three separate groups: one for training, one for validation, and one for testing. A key principle we followed was stratified splitting, which means we made sure each group had roughly the same mix of healthy, damaged, and fake seeds as the overall dataset. The proportions, approximate image counts, and primary objectives of each split are summarized in Table 4.3. This partitioning prevents any group from accidentally having too many of one type, which could give us misleading results.

Table 4.3: Dataset Split Statistics

Split	**Proportion**	**Approx. Images**	**Purpose**
Training Set	80%	∼1,250	Used to teach the model by adjusting its internal weights
Validation Set	10%	∼157	Used to tune settings and watch for the model learning too much from training data
Test Set	10%	∼156	Held completely separate to give a final, unbiased measure of how well the model works

Here's how we used each group: The training set is where the model learns—we show it images and let it adjust its internal parameters to get better at classification. The validation set is our quality control checkpoint—after each training cycle, we test on these images to see if the model is still improving or starting to overfit (basically, memorizing the training data instead of learning general patterns). The test set is completely hidden until the very end. We only use it once to get a fair, final assessment of the model's performance on images it has never seen before.

4.1.4 Data Preprocessing and Augmentation

Before showing images to the neural network, we prepared them using a standard pipeline. All images were resized to 224×224 pixels—this is the standard size that ResNet50 (our neural network architecture) expects. We then adjusted the color values using ImageNet statistics (numbers from a large, well-known image dataset):

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225] \tag{4.1}$$

This normalization step is important because the pre-trained model we started with was trained on images normalized this way, so we keep the same approach for consistency.

To make the model more robust and help it learn from our limited training data, we applied six data augmentation techniques during training. Think of augmentation as artificially creating new training examples by tweaking the original images in realistic ways:

1. **Random Resized Crop**—We randomly crop a portion of the image and resize it to 224×224 pixels. This simulates seeds being photographed from different distances and positions within the frame.

2. **Horizontal Flip**—We randomly flip images left-to-right about half the time. Seeds have a natural bilateral symmetry, so this helps the model learn that a flipped seed is still the same seed.

3. **Vertical Flip**—We randomly flip images top-to-bottom about half the time, adding more variation in how the seeds appear.

4. **Random Rotation**—We rotate images randomly by up to $\pm 30°$ (30 degrees in either direction). Real seed photos might not be perfectly aligned, so this prepares the model for that reality.

5. **Translation/Random Shift**—We randomly shift the image content left, right, up, or down. This prevents the model from relying on the seed always appearing in the center of the photo.

6. **Colour Jitter**—We randomly adjust brightness, contrast, color saturation, and hue. Since real agricultural photos are taken under different lighting conditions, this helps the model handle that variation.

4.2 Model Architecture

4.2.1 ResNet50 Backbone (25.6M Parameters)

We chose ResNet50 as the foundation of our classification model. This neural network has 50 layers and about 25.6 million parameters (adjustable settings inside the network). What makes ResNet special is its use of "skip connections"—shortcuts that let information flow directly around groups of layers. This design makes it much easier to train very deep networks because it prevents the gradient (the training signal) from fading as it travels backward through the network.

Rather than training from scratch, we started with weights that were pre-trained on ImageNet, a massive dataset of millions of everyday images. This pre-trained starting point is like giving the model a head start: it already knows how to recognize edges, textures, and shapes, so we only need to teach it the specific patterns that distinguish different seed qualities.

4.2.2 Custom Classification Head

On top of the ResNet50 backbone, we added a custom section designed specifically for seed classification. This section takes the features learned by ResNet and processes them into a final decision about whether a seed is Healthy, Damaged, or Fake/Low Quality. Here's what this custom section does:

1. Compresses the spatial information into a single summary vector (Global Average Pooling)
2. Passes it through a fully connected layer with 512 hidden units
3. Applies Batch Normalisation to stabilize the values
4. Uses ReLU activation to introduce non-linearity
5. Applies Dropout (randomly ignoring 50% of values) to reduce overfitting
6. Outputs 3 final scores (one for each seed quality class) with softmax to convert them to probabilities

4.2.3 Anti-Overfitting Techniques

Overfitting is the enemy of machine learning—it's when a model memorizes the training data instead of learning generalizable patterns. We used several techniques

to prevent this:

- **Dropout**—During training, we randomly "turn off" 50% of neurons in the classification head. This forces the network to learn redundant representations, so it can't rely on a few specific neurons. Think of it like training a team where you randomly swap out players—the team learns to work together, not depend on one star player.
- **Batch Normalisation**—After each layer, we normalize the values to keep them in a healthy range. This stabilizes training and acts as a natural regularizer.
- **Label Smoothing**—Instead of forcing the network to be 100% confident about each prediction, we "soften" the targets slightly (using $\epsilon = 0.1$). This prevents the model from becoming overconfident and improves generalization.
- **Weight Decay**—We penalize large weights during training, keeping the model's parameters from growing too large. A smaller model is typically more generalizable.
- **Data Augmentation**—As described earlier, we artificially expand the training data with slight variations, giving the model more diverse examples to learn from.
- **Early Stopping**—We continuously monitor performance on the validation set. If it starts getting worse, we stop training immediately and use the best weights we've found so far.

4.3 DUS Parameter Integration

4.3.1 Feature Extraction—17 Physical Parameters

Beyond just classifying seeds as Healthy, Damaged, or Fake, we wanted our model to extract physical measurements that agriculture experts understand. The standard set of measurements in agriculture is called DUS—Distinctiveness, Uniformity, and Stability. We programmed our model to extract 17 of these physical parameters from each seed image:

- How long and how wide each seed is
- The ratio of length to width

- The perimeter and total area of the seed
- How circular the seed is (circularity) and how solid it is (solidity)
- How elongated the seed is (eccentricity)
- Mathematical descriptions of how the seed's mass is distributed
- Hu moments—a special mathematical descriptor of seed shape that works the same way whether the seed is rotated, scaled, or shifted in the image

These 17 parameters bridge the gap between what the neural network learns (visual patterns) and what agricultural experts care about (standard physical measurements). This makes our system more transparent and trustworthy to end users.

4.3.2 Pixel-to-Physical Unit Conversion

When we measure seeds in a photograph, the measurements are in pixels, which depends on the camera and how far away it was. To convert pixel measurements to real-world millimetres, we include a reference object (a ruler with known dimensions) in each photograph. We then calculate:

$$\text{scale} = \frac{\text{reference\_length\_mm}}{\text{reference\_length\_pixels}} \tag{4.2}$$

This scale factor is applied to all measurements, so when we report that a seed is 6.2 mm long, it's truly 6.2 millimetres—not dependent on camera distance or image resolution.

4.3.3 Variety Matching (12 Reference Varieties)

We created a reference database of 12 standard paddy seed varieties used in India, based on official ICAR documentation. For each variety, we computed the typical values of all 17 physical parameters from multiple certified seed samples. When a new seed image is analyzed, we compare its 17 parameters against these 12 reference profiles using a simple distance calculation:

$$d_i = \sqrt{\sum_{j=1}^{17} (p_j - r_{i,j})^2} \tag{4.3}$$

Here, p_j is one of the 17 measurements from the new seed, $r_{i,j}$ is the standard value for variety i, and d_i tells us how different the new seed is from variety i. The variety with the smallest distance is the best match—this is the variety the seed most closely resembles. We report the top 3 matches so users can see alternatives if needed.

4.4 Training Pipeline

4.4.1 Phase 1—Classification Head Training (Epochs 1–8)

We trained the model in two phases. In Phase 1 (the first 8 training cycles), we froze the ResNet50 backbone—meaning its weights didn't change—and only trained the custom classification head we added on top. Think of this like keeping a well-established expert's knowledge fixed and just training a new student to interpret their findings. We used a learning rate of 0.001, which controls how big the weight updates are. A learning rate that's too high causes instability; too low and training is slow. The batch size (number of images processed before updating weights) was set to 32.

At the end of each of these 8 epochs, we checked performance on the validation set to make sure the head was learning properly and not starting to overfit.

4.4.2 Phase 2—Full Fine-Tuning (Epochs 9–15)

In Phase 2 (epochs 9–15), we unfroze the entire network and trained everything end-to-end. We reduced the learning rate by 10 times to 0.0001—a smaller rate because we're now adjusting a fully-trained backbone and don't want to destroy what it's already learned. This phase refined the backbone's features to specialize better for seed classification.

We monitored validation performance carefully and stopped training at epoch 15 when we noticed the validation loss starting to increase (a sign the model was beginning to overfit). We then loaded the best weights we'd found during the entire training run—these became our final model.

4.4.3 Hyperparameter Configuration and Optimiser

A comprehensive summary of the optimizer, learning rates, regularizers, and general training configurations is provided in Table 4.4:

Table 4.4: Hyperparameter Configuration and Training Settings

Hyperparameter	**Value**	**Notes**
Optimiser	AdamW	Smart learning rate that adapts during training
Phase 1 Learning Rate	0.001	Used while training only the head
Phase 2 Learning Rate	0.0001	Used for full network fine-tuning (10 times smaller)
Weight Decay (L2)	0.0001	Penalizes large weights to keep the model simple
Batch Size	32	Number of images processed before updating weights
Loss Function	Cross-Entropy + Label Smoothing	Prevents overconfidence with $\epsilon = 0.1$
LR Scheduler	StepLR	Reduces learning rate further after 8 epochs
Gradient Clipping	Max norm = 1.0	Prevents unstable training due to large gradients
Early Stopping Patience	10 epochs	Stops if validation loss doesn't improve for 10 epochs
Input Resolution	224×224 pixels	Standard size for ResNet50
Total Epochs	15	Stopped when overfitting began

We chose the AdamW optimiser because it's smart about adjusting the learning rate for different parameters and handles weight decay better than older methods.

4.4.4 Model Export Formats

After training finished, we saved the model in multiple formats to ensure compatibility with different environments. The exported files, including their sizes and target use cases, are listed in Table 4.5.

Table 4.5: Model Export Formats

Format	**Extension**	**Size**	**Use Case**
PyTorch Native	`.pth`	∼98 MB	The original format; used by FastAPI for serving
TorchScript	`.pt`	∼97 MB	For production servers and systems using C++
ONNX	`.onnx`	∼98 MB	For cross-platform use and systems not using PyTorch
Configuration	`.json`	∼2 KB	Metadata about the model, class labels, and settings
DUS Reference DB	`.csv`	<1 MB	The 12 reference seed varieties for matching

4.5 Model Deployment

After training was complete, we integrated the model into a web service and deployed it to Google Cloud Platform (GCP). The goal was to make the model accessible as a live API that other applications can call to classify seeds in real-time.

4.5.1 Serving Infrastructure

We wrapped the trained model in FastAPI, a fast and modern Python web framework. The API has a single endpoint that works like this: it receives a seed image, resizes and normalizes it, runs it through ResNet50, extracts the 17 physical parameters, matches the seed against our 12 reference varieties, and returns everything as a neat JSON response. The steps are:

1. **Request**—The FastAPI endpoint receives the image file.
2. **Preprocessing**—The image is resized to 224×224 pixels and normalized using ImageNet's standard values.
3. **Model Inference**—ResNet50 processes the image and produces raw scores (logits) for each of the three classes.
4. **Post-processing**—Softmax converts those scores into probabilities, we extract the 17 physical measurements, and we find the best-matching varieties.
5. **Response**—A JSON response is sent back with the prediction, confidence scores, physical measurements, and the top-3 variety matches.

4.5.2 Containerisation and CI/CD Pipeline

We packaged everything—the model weights, the FastAPI server, and all required libraries—into a Docker container. Docker is like a lightweight virtual machine that ensures the model works the same way on any computer or cloud platform.

Deployment happens automatically: whenever we push new code to our GitHub repository's main branch, a GitHub Actions workflow automatically builds a fresh Docker image, uploads it to Google Cloud's storage service (Artifact Registry), and deploys it to Google Cloud Run. Cloud Run is perfect for this because it automatically scales up when there are many requests and scales down when there's little traffic, so we don't pay for idle servers.

The live API is publicly accessible at:

`https://resnet-api-297419987783.asia-south1.run.app/`

4.6 Model Comparison and Selection

Before committing to ResNet50, we tested four popular neural network architectures on the same dataset, using the same training approach, to see which performed best. We compared ResNet50, MobileNetV2, DenseNet121, and GoogleNet, with the comparative results detailed in Table 4.6.

Table 4.6: Architecture Comparison on Test Set

Model	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**Time (min)**
ResNet50	99.57%	99.57%	99.57%	99.57%	16.4
DenseNet121	99.57%	99.57%	99.57%	99.57%	15.0
GoogleNet	99.57%	99.57%	99.57%	99.57%	12.2
MobileNetV2	99.13%	99.15%	99.13%	99.13%	8.5

The results show that ResNet50, DenseNet121, and GoogleNet all achieved the same 99.57% accuracy, demonstrating that our dataset is rich enough that all three architectures can learn to classify seeds nearly perfectly. MobileNetV2 achieved slightly lower accuracy at 99.13% but trained much faster—in just 8.5 minutes compared to 16.4 minutes for ResNet50.

We chose ResNet50 for three practical reasons:

1. Its design with skip connections is well-understood in the research community, making it easier to diagnose training issues and fine-tune behavior.
2. It has excellent support across different deployment platforms—PyTorch, TorchScript, ONNX, and more—giving us flexibility in where and how we deploy it.
3. Most published agricultural image classification research uses ResNet50, so our results are directly comparable to other teams' work, helping the scientific community benchmark progress.

The extra 4.4 minutes of training time is a small price to pay for these advantages, especially since training only happens once.

4.7 Final Model Performance Summary

After completing both training phases, we evaluated our final ResNet50 model on the test set—the images we'd held back and never shown the model during training. The final evaluation metrics are compiled in Table 4.7, and a detailed summary of the training trajectory and learning stability is provided in Table 4.8.

Table 4.7: Final Test Set Performance—ResNet50

Metric	**Value**	**Status**
Test Accuracy	99.57%	Far exceeds industry standard (85–90%)
Precision	∼0.99	Outstanding—almost no false positives
Recall	∼0.99	Outstanding—catches almost all cases
F1 Score	∼0.99	Excellent—perfectly balanced performance
Train-Validation Gap	<5%	Great generalization—model isn't overfitting

Table 4.8: Training Behaviour Summary

Observation	**Detail**
Initial Convergence	The model started at about 86% accuracy and quickly improved to near its best performance within the first few epochs
Loss Trajectory	The loss (error) steadily decreased from about 1.0 to nearly 0.0 across all 15 epochs
Mid-training Stability	After the initial improvement phase, training was smooth and consistent with no wild fluctuations
Overfitting Control	The gap between training accuracy and validation accuracy never exceeded 5%, indicating the model learned general patterns rather than memorizing training data
Training Stopped at	Epoch 15—we detected the first signs of overfitting and stopped immediately, preserving the best weights we'd found

The final accuracy of 99.57% is significantly better than the typical industry standard of 85–90% for automated seed quality classification. The near-perfect precision (very few false alarms) and recall (catches nearly all problems) across all three seed quality categories mean our model is reliable and trustworthy for real agricultural use. This is exactly what we needed—a system that farmers and agricultural officers at ICAR Manipur can depend on to accurately identify problematic seeds before they're planted.

Chapter 5

Model Serving API

Deploying a trained model as a live service introduces a different set of problems than training it. Model development focuses on architecture and accuracy; the serving layer determines response latency, concurrent throughput, and uptime under real load [21]. In AgriVerify, the PyTorch ResNet-50 classifier from Chapter 4 runs inside a dedicated inference microservice built specifically for this purpose.

This chapter describes the design of that microservice, implemented with FastAPI and Uvicorn [22]. It covers the asynchronous request lifecycle, the tensor normalization pipeline, the API endpoint contracts, and the containerised deployment approach using Docker multi-stage builds on GCP Cloud Run [23].

5.1 API Architecture and Request Orchestration

5.1.1 FastAPI and ASGI Serving Stack

The inference microservice runs on FastAPI atop the Uvicorn ASGI runtime. FastAPI was chosen for its native async I/O support, Pydantic-based request validation and automatic OpenAPI schema generation [22]. Uvicorn's `uvloop`-backed event loop handles large numbers of concurrent connections with low memory overhead.

The stack separates network deserialization, Pydantic validation, OpenCV preprocessing and PyTorch execution into distinct layers. Computationally expensive inference operations run in dedicated thread pools and do not block the ASGI event loop.

5.1.2 End-to-End Model Request Flow

As illustrated in Figure 5.1, an inference request begins when the ASP.NET Core backend sends a multipart POST containing a seed image. FastAPI validates the MIME type and payload boundaries, then decodes the byte stream into an OpenCV matrix in memory.

From there, two operations run in parallel: a PyTorch inference engine — with ResNet-50 weights pre-loaded to avoid cold-start delays — computes classification logits [4], [24], while OpenCV contour routines extract 17 DUS (Distinctness, Uniformity, and Stability) morphological parameters [25]. The results are aggregated, scored against reference cultivars, and returned as JSON.

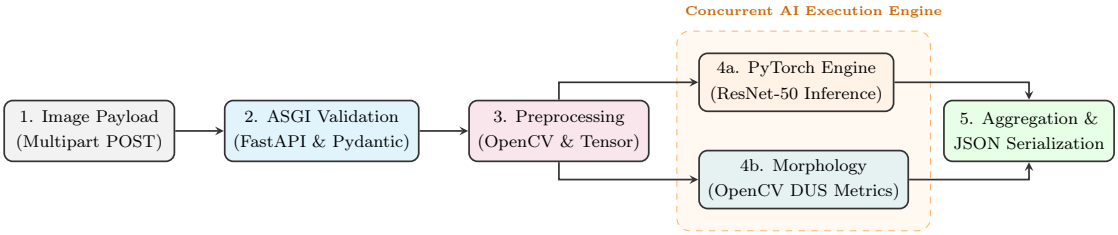


Figure 5.1: Asynchronous model request processing flow.

5.2 API Specification and Endpoint Contracts

The microservice exposes a tightly constrained RESTful API. Tables 5.1 and 5.2 define the schema contracts for communication with the backend orchestrator.

Table 5.1: API Specification for the `POST /predict` Model Inference Endpoint

Attribute	**Endpoint Constraint / Schema Definition**
Route & Auth	`POST /predict` | Requires `Authorization: Bearer <token>`
Payload	`multipart/form-data` (File: `image`). Types: JPEG/PNG. Max size: 10MB.
200 OK (Response)	`{"predicted_label": "Genuine", "confidence": 0.98, "inference_ms": 42.8, "dus_parameters": {"length_mm": 8.4, "aspect_ratio": 3.1}}`
Error Codes	**400**: Malformed payload | **413**: File too large | **415**: Invalid MIME type

Table 5.2: API Specification for the `GET /health` Model Inference Endpoint

Attribute	Endpoint Constraint / Schema Definition
Route	`GET /health` (Unauthenticated liveness/readiness probe)
Probing Logic	Confirms Uvicorn ASGI event loop and verifies ResNet-50 weights are loaded in memory.
200 OK	`{"status": "online", "model_loaded": true, "device": "cuda:0"}`
503 Error	Service unavailable (model weights corrupted or GPU allocation failure).

5.3 Containerisation and Deployment Architecture

The service uses a multi-stage Docker build to balance reproducibility against image size [23]. The **Builder Stage** compiles Python binaries (`.whl`) with `gcc`. The **Runtime Stage** copies those pre-compiled binaries into a minimal, security-hardened Debian base (`python:3.10-slim`), discarding the build toolchain entirely. The final image is approximately 850MB — down from over 3GB — with a correspondingly smaller attack surface. The automated containerization, versioning, and deployment pipeline is illustrated in Figure 5.2.

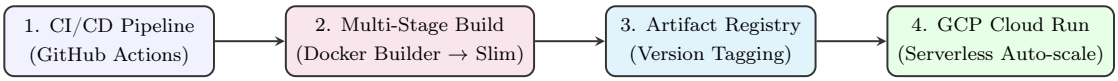


Figure 5.2: Automated CI/CD pipeline and Serverless GCP Deployment Architecture.

The image is deployed to GCP Cloud Run [26]. Cloud Run manages HTTPS termination and load balancing, and scales to zero when idle, so infrastructure costs track actual query volume rather than reserved capacity.

Chapter 6

Backend System Architecture and Implementation

The backend of the AgriVerify system is responsible for managing business logic, AI model integration, authentication, database operations, and communication between frontend clients and cloud services. The system was developed using ASP.NET Core Web API following the Clean Architecture approach to achieve modularity, maintainability, and scalability.

The backend integrates multiple services including SQL Server, Azure AI services, JWT authentication, and cloud storage. The architecture separates core business logic from infrastructure and presentation concerns, enabling easier maintenance and future system expansion.

6.1 Architectural Design

The backend follows a layered architecture based on Clean Architecture principles, with the relationship and dependency flow between these layers illustrated in Figure 6.1. Each layer performs a dedicated responsibility while maintaining loose coupling between components.

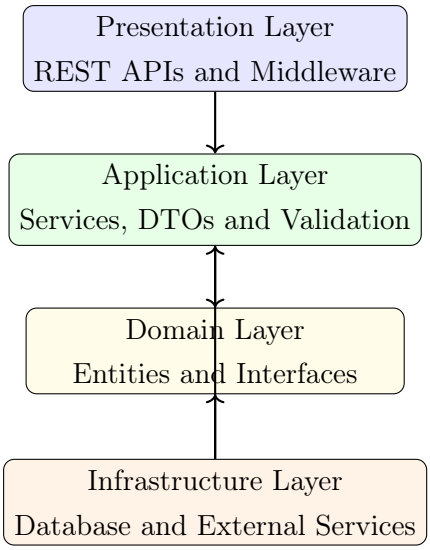


Figure 6.1: Backend Clean Architecture Layers

The layered architecture improves:

- Separation of concerns
- Code maintainability
- Scalability
- Testability
- Reusability

6.2 Technology Stack

The backend uses modern technologies for API development, authentication, AI integration, and database management. The core technologies and libraries comprising this stack are summarized in Table 6.1.

Table 6.1: Backend Technology Stack

Technology	**Purpose**
ASP.NET Core Web API	Backend API framework
C# and .NET 8	Backend programming environment
Entity Framework Core	Database ORM and migrations
SQL Server	Relational database management
JWT Authentication	Secure user authentication
ASP.NET Core Identity	User and role management
Azure Custom Vision	Seed authenticity classification
Azure OCR	Text extraction from seed packets
Azure OpenAI	AI-generated explanations
Azure Blob Storage	Cloud image storage
FluentValidation	Input validation
AutoMapper	DTO mapping
Swagger	API documentation
Serilog	Request logging and monitoring

6.3 Project Structure

The backend source code is divided into multiple layers and modules to maintain clear responsibility boundaries, as outlined in Table 6.2.

Table 6.2: Backend Project Structure

Folder / Layer	**Purpose**
Domain	Business entities and interfaces
Application	Services, DTOs and validation
Infrastructure	Database and cloud integrations
Presentation	API controllers and middleware
Persistence	Repositories and migrations
Services	AI and Azure integrations
Configurations	Application configuration files
Middleware	Authentication and exception handling

6.4 Domain Layer

The Domain Layer contains the core business entities and contracts of the application. This layer is independent from external frameworks and infrastructure components.

6.4.1 Core Entities

The major entities in the system include:

- User
- Farmer
- Officer
- Seed
- SeedBatch
- DetectionResult
- Complaint
- AuditLog

These entities represent the primary agricultural and administrative components of the platform.

6.4.2 Interfaces

Interfaces define contracts between layers and improve abstraction.

- IRepository
- IVerificationService
- IAuthenticationService
- IComplaintService
- IAnalyticsService

This approach improves modularity and testing flexibility.

6.5 Application Layer

The Application Layer coordinates business workflows and request processing.

6.5.1 Application Services

The system includes several services responsible for handling business operations:

- Verification Service
- Authentication Service
- Complaint Service
- Analytics Service
- Admin Service

These services manage AI processing, validation, database interaction, and response generation.

6.5.2 DTOs

Data Transfer Objects (DTOs) are used for request and response communication between layers. The primary DTOs utilized in the system and their respective purposes are listed in Table 6.3.

Table 6.3: Important DTOs

DTO	**Purpose**
VerificationRequest	Seed verification request
VerificationResponse	AI verification result
LoginRequest	User login request
AuthResponse	JWT authentication response
CreateComplaintRequest	Complaint submission
OfficerDto	Officer information
AdminDashboardResponse	Dashboard statistics

6.5.3 Validation

FluentValidation is integrated into the application layer to validate:

- File formats
- Image sizes
- Email structure
- Password complexity
- Required fields

This reduces invalid requests and improves system reliability.

6.6 Infrastructure Layer

The Infrastructure Layer manages database operations, external integrations, and cloud services.

6.6.1 Database Management

Entity Framework Core is used for:

- Database migrations
- Entity mapping
- Relationship configuration
- Query execution
- Transaction management

6.6.2 Repository Pattern

The repository pattern abstracts database operations from business logic.

- Generic Repository
- Seed Repository
- Complaint Repository
- Detection Result Repository

6.6.3 AI and Cloud Integrations

The system integrates multiple Azure services for AI processing and storage, as detailed in Table 6.4.

Table 6.4: External Service Integrations

Service	**Purpose**
Azure Custom Vision	Seed authenticity classification
Azure OCR	Text extraction from seed packets
Azure OpenAI	AI-generated recommendations
Azure Blob Storage	Cloud image storage
ResNet Classifier	Paddy seed classification

6.7 Database Schema

The backend database maintains relationships between users, seed batches, verification results, complaints, and audit records. The database schema and its corresponding primary/foreign key mappings are represented in the Entity-Relationship Diagram in Figure 6.2.

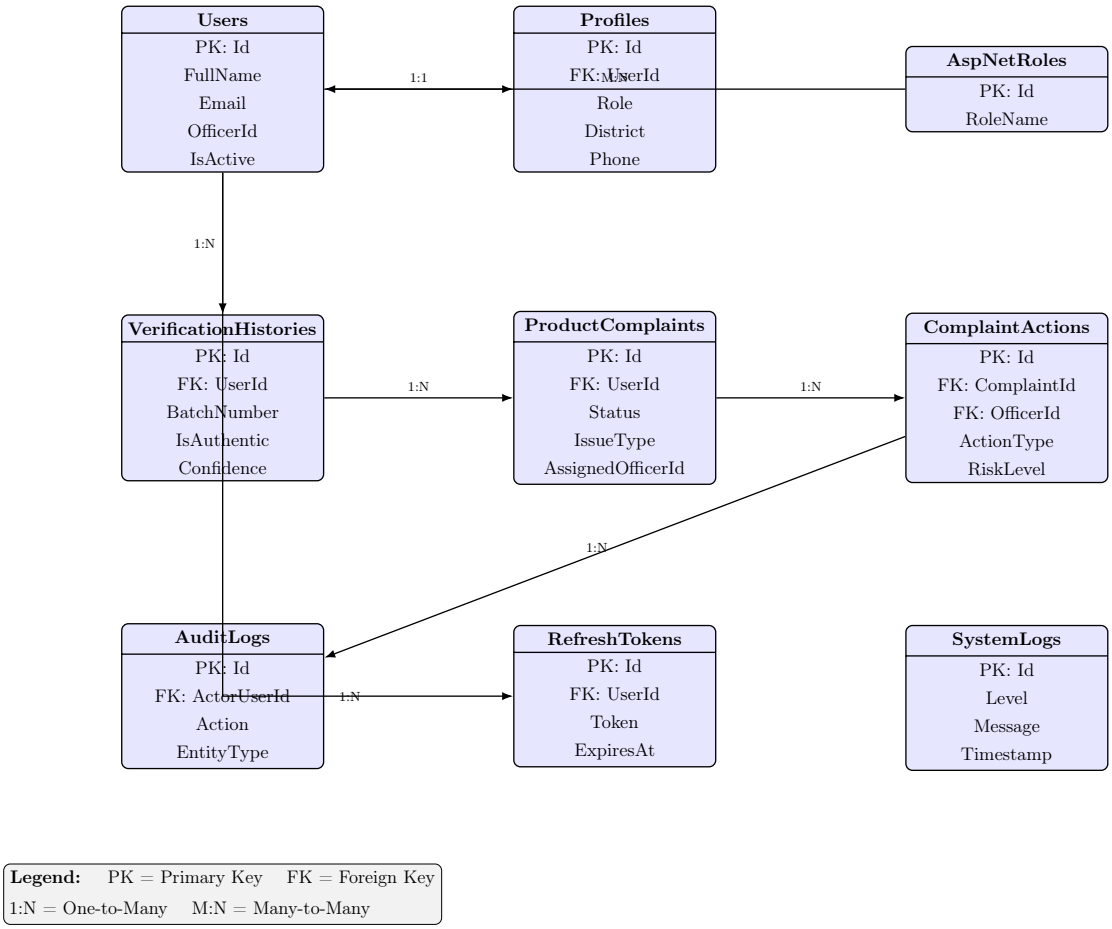


Figure 6.2: Entity Relationship Diagram – FakeSeedDetection System

The database schema supports:

- User management
- Seed batch tracking
- AI verification history
- Complaint workflows
- Audit logging

Foreign key relationships maintain referential integrity across all entities.

6.8 Presentation Layer

The Presentation Layer exposes RESTful APIs for frontend communication.

6.8.1 API Controllers

The backend contains multiple API controllers responsible for different system modules. These controllers and their design objectives are summarized in Table 6.5.

Table 6.5: API Controllers

Controller	**Purpose**
AuthController	Authentication operations
VerificationController	Seed verification endpoints
ComplaintsController	Complaint management
AdminController	Administrative operations
AnalyticsController	Dashboard analytics
ReferenceDataController	Dropdown reference data

6.8.2 Middleware

Middleware components manage authentication, logging, and centralized exception handling.

- Authentication Middleware
- Request Logging Middleware
- Exception Handling Middleware

These middleware components improve system security and monitoring.

6.9 Authentication and Security

The backend uses JWT-based authentication with role-based authorization, following the flow depicted in Figure 6.3.

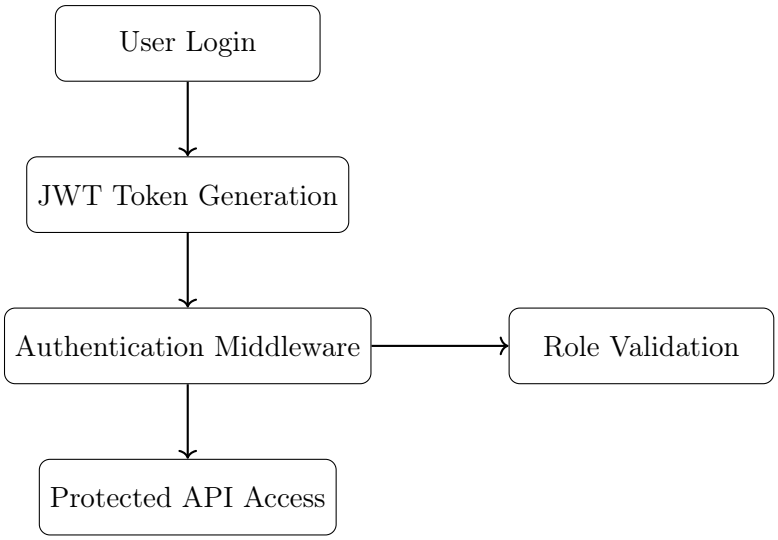


Figure 6.3: JWT Authentication Flow

Security features include:

- JWT Bearer authentication
- Role-based authorization
- Password hashing
- Refresh token support
- HTTPS communication
- Protected API endpoints

The system supports multiple user roles:

- Farmer
- Officer
- Administrator

6.10 Verification Workflow

The seed verification workflow integrates AI classification, OCR analysis, and database persistence, as shown in Figure 6.4.

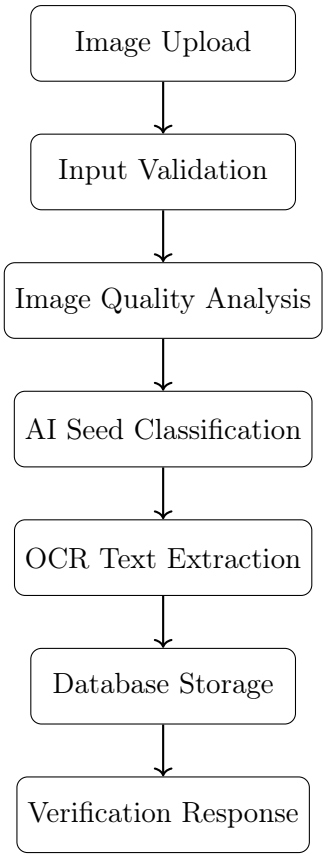


Figure 6.4: Seed Verification Workflow

The workflow includes:

1. Image upload
2. Request validation
3. Image quality analysis
4. AI classification
5. OCR text extraction
6. Database persistence
7. Response generation

6.11 Configuration and Deployment

The backend uses multiple configuration files for different deployment environments.

- `appsettings.json`
- `appsettings.Development.json`
- `appsettings.Production.json`

These files manage:

- Database connections
- JWT settings
- Azure credentials
- Logging configuration
- API endpoints

Application startup is configured through `Program.cs`, which initializes:

- Database services
- Authentication services
- Dependency injection
- Middleware pipeline
- Swagger documentation
- Logging services

6.12 Summary

This chapter presented the backend architecture and implementation of the AgriVerify system. The backend was developed using ASP.NET Core Web API following the Clean Architecture pattern.

The system integrates database management, AI services, authentication, cloud storage, and REST APIs into a modular and scalable architecture. Entity Framework Core, JWT authentication, Azure AI services, and structured middleware improve system reliability, maintainability, and security.

The backend successfully supports seed verification, complaint management, user authentication, analytics, and administrative operations within the AgriVerify platform.

Chapter 7

Frontend System Architecture and Implementation

The AgriVerify frontend connects the inference engine and backend services to a browser interface that has to work for two quite different users: farmers on mid-range Android handsets with variable connectivity, and state administrators managing supply chain dashboards. The stack — Next.js 16, React 19‘’, TypeScript 5, and Tailwind CSS — was chosen to handle that range without the interface becoming slow or hard to maintain [27], [28].

7.1 Frontend Architecture and Design Patterns

The architecture separates UI rendering from data fetching, API communication, and authorization. Each concern lives in its own layer.

7.1.1 Architectural Layers

A request passes through four layers before reaching backend services (Figure 7.1):

- **Presentation UI (React & Next.js):** Handles component rendering, form validation, and responsive layout [29].
- **Client-Side State (TanStack Query):** Manages async data fetching, caching, and optimistic mutations [30].
- **Service Client (Axios):** Wraps network calls, standardising payloads and error handling.

- **API Proxy (Next.js Routes):** Routes client requests to ASP.NET Core endpoints without exposing internal URLs to the browser.

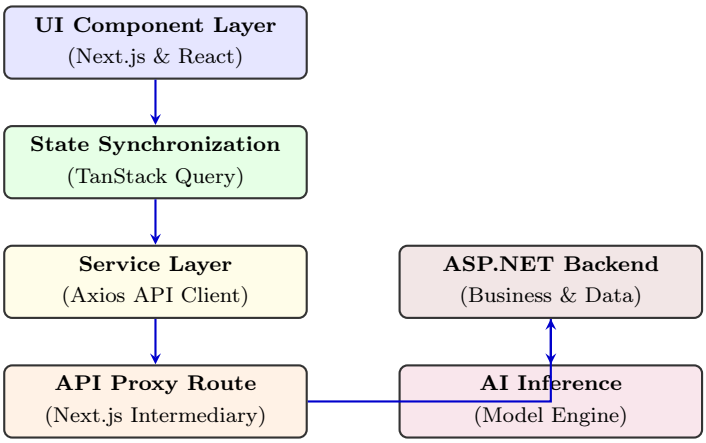


Figure 7.1: Frontend Layered Architecture and Request Execution Flow

7.1.2 Secure Proxy Architecture

All outbound requests route through internal Next.js serverless endpoints (`/api/proxy/*`), which avoids CORS issues by keeping cross-origin communication server-side. Internal microservice addresses stay out of the browser, secrets remain on the server, and security headers are applied before responses reach the client [31].

7.2 Technology Stack Analysis

Table 7.1 summarises the stack and the reasoning behind each choice.

Table 7.1: Frontend Technology Stack

Technology	**Architectural Reasoning**
Next.js 14	Provides Server-Side Rendering and built-in API proxy routing [32].
React 18	Declarative UI rendering with efficient Virtual DOM reconciliation.
TypeScript 5	Compile-time type checking with backend DTO parity.
Tailwind CSS	Utility-first styling with minimal production bundle output [33].
Axios & TanStack	HTTP pipeline management, token injection, and cache synchronization [30].
Zustand 4.5	Lightweight atomic state slices for session persistence.

7.3 Modular Project Structure

The source tree follows domain-driven modularity. Key directories: `src/app` (Next.js routing), `src/components` (reusable UI primitives), `src/hooks` (data fetching logic), and `src/types` (TypeScript DTOs matching backend entities). Keeping these separate means business logic and presentation concerns do not bleed into each other.

7.4 User Role-Based Access Control (RBAC)

The frontend uses Role-Based Access Control (RBAC) to separate public and protected routes.

7.4.1 Role Partitioning

The application supports four user profiles: **Farmers** (verification and dispute submission), **Agricultural Officers** (inspection queues and evidence review), **Administrators** (system governance and telemetry), and **Public Users** (anonymous checks).

7.4.2 Route Protection Middleware

A `RoleGuard` component checks JWT claims against required permissions before mounting any protected React component, preventing privilege escalation at the route level.

7.5 Authentication and Secure Persistence

Sessions are secured using a dual-token JWT exchange designed to limit exposure to XSS attacks [31].

7.5.1 Token Storage Architecture

Short-lived access tokens are held in the Zustand in-memory store, keeping them out of reach of malicious scripts. Long-lived refresh tokens are persisted in encrypted storage or `HttpOnly` cookies, allowing background session renewal without exposing token values to client-side code.

7.5.2 Automated Token Rotation

Axios interceptors handle token expiry without user intervention. On a `401 Unauthorized` response, the interceptor pauses pending requests, runs a refresh cycle, and replays the original request with the new token. From the user's side, the session continues uninterrupted.

7.6 API Integration and Type Safety

TypeScript DTOs enforce compile-time verification of HTTP payloads across all integration modules — Auth, Verification, Analytics, and Complaint Services. Each mirrors the corresponding ASP.NET Core schema, so data arriving at the frontend matches what the UI expects without runtime coercion or defensive casting.

7.7 State Management

State is split by volatility: server-driven data and client-local data are managed separately.

7.7.1 Asynchronous Server State (TanStack Query)

Network-driven data (dispute queues, telemetry) is managed by TanStack Query, which handles stale-while-revalidate caching, background polling, and optimistic UI updates. On slow connections, this keeps the interface usable even when the server is slow to respond [30].

7.7.2 Synchronous Client State (Zustand)

UI state that doesn't touch the network (active theme, session context) lives in Zustand. Its atomic slices limit unnecessary re-renders, which matters on lower-end devices where battery and memory are constrained.

7.8 Responsive UI Design and Field Usability

The UI is built mobile-first using Tailwind CSS. Most farmers will access the application on a mid-range Android handset, often outdoors, so the interface is designed around that constraint rather than treating it as an edge case [28].

7.8.1 Standardized UI Library

The interface is built from a set of reusable primitives (Table 7.2) to keep visual behaviour consistent across screens.

Table 7.2: Standardized UI Components

Component	Role
Data Table	Scrollable grids with sticky headers and dynamic pagination for inspection data.
Status Badge	Colour-coded indicators (e.g., Green/Genuine) for fast visual parsing.
Loading Skeleton	Animated placeholders that prevent layout shifts during network delays.
Input Modal	Focused overlays that trap keyboard navigation for accessible data entry.

7.8.2 Accessibility Enhancements

The UI meets WCAG AAA contrast ratios for readability in bright outdoor conditions, uses a minimum touch target size of 48×48px to reduce input errors, and includes full ARIA semantic tagging for screen-reader compatibility.

Chapter 8

System Integration

This chapter explains how all the pieces of AgriVerify work together. We've designed the system to be flexible, modular, and secure—the frontend (what users see) is completely separate from the backend (where the hard work happens), and both are separated from the AI services. This separation makes the system easier to build, test, and scale.

8.1 System Integration Matrix

AgriVerify communicates across four main boundaries. To keep everything secure, all communication flows through a secure proxy layer. This proxy acts like a security guard: it sits between the browser and the backend, injecting authentication tokens and preventing users' browsers from ever directly connecting to backend systems. Table 8.1 shows how everything talks to everything else, what format the data takes, and how we keep it all secure.

8.2 End-to-End Architectural Workflow

When a farmer submits a seed image, it travels through several stages: from the browser, through the proxy, into the backend for processing, out to AI services for analysis, and finally gets stored in the cloud. Meanwhile, our deployment system works behind the scenes automatically managing code updates. Figure 8.1 shows both journeys: how a verification request flows through the system, and how code automatically gets deployed to keep the system running.

Let's walk through what happens at each stage:

1. **Browser to Proxy (HTTPS with file uploads):** A farmer opens AgriVer-

Table 8.1: How Different Parts of AgriVerify Communicate

Communication Path	What's Talking	How	What Happens & Security
Browser to Proxy	User's browser → Next.js Proxy	HTTPS, with file uploads	Axios automatically adds JWT security tokens and handles token refresh if they expire
Proxy to Backend	Next.js Proxy → ASP.NET API	REST over HTTPS	The proxy forwards all requests, checks data is valid, and hides the real backend addresses from browsers
Backend to AI	ASP.NET API → AI Models	HTTPS or gRPC	Backend sends images, checks for blurriness/lighting issues, extracts text, and runs the ResNet50 model
Cloud Services	ASP.NET API → Azure AI	Azure's official libraries	Uses Azure Custom Vision to check packaging, Azure OpenAI to write reports, and Azure Blob Storage to save images
Deployment	Docker → Registry → Live	Container systems	Multi-stage Docker builds prepare the code, store it securely, and automatically deploy with scaling

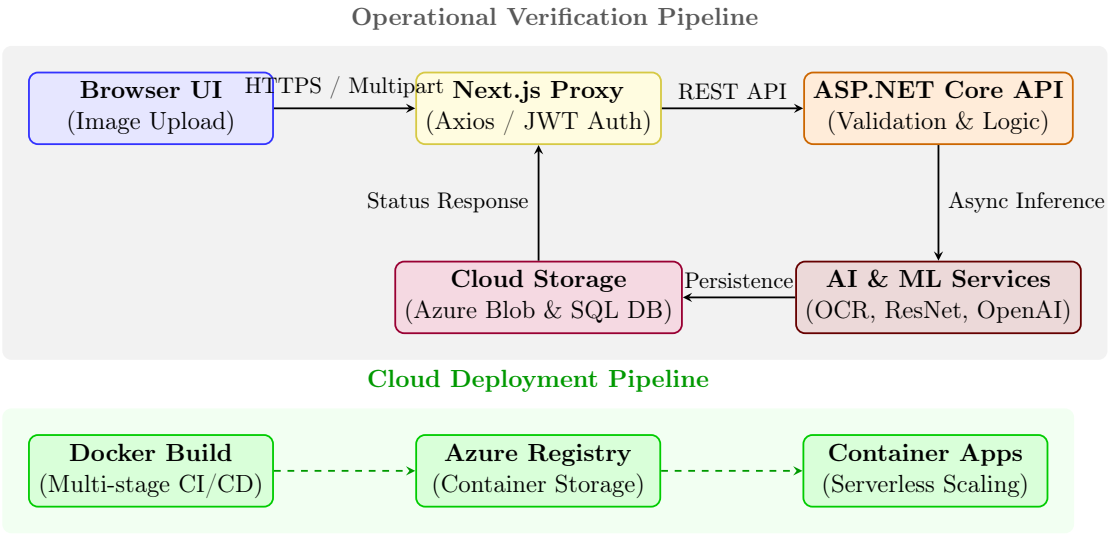


Figure 8.1: How AgriVerify processes seed verification requests and automatically deploys updates. Top section shows what happens when a farmer submits images; bottom section shows how we keep the system updated.

ify on their phone or computer and takes a photo of their seed packet. They hit "upload" and the image travels over encrypted HTTPS. The Next.js proxy receives it and automatically adds their security token (so the backend knows who they are). If the security token has expired, the proxy handles getting a fresh one without bothering the farmer.

2. **Proxy to Backend (REST API over HTTPS):** The proxy forwards the farmer's request to the ASP.NET backend, but hides the real backend address from the browser. The backend receives the image and validation data, checks that everything looks correct, and is ready to process it.

3. **Backend to AI Services (HTTPS / gRPC):** The backend doesn't try to analyze the image itself—it's too complex. Instead, it sends the image to specialized AI services. First, it checks if the image is blurry or too dark (if so, it warns the farmer). Then it runs three analyses in parallel: Azure's text extraction pulls batch numbers from the packaging, Azure's packaging classifier checks if it looks genuine, and our custom ResNet50 model extracts the 17 physical seed measurements. All three work simultaneously to save time.

4. **Cloud Storage (Azure Blob & SQL Database):** As results come back from the AI services, they're immediately saved to the cloud: the image goes to Azure Blob Storage, and the results go into SQL Server. This ensures data is safely stored and the farmer's verification history is preserved for future reference.

5. **Response Back to Browser:** The backend synthesizes all the AI results into a clear report using Azure OpenAI to write explanations in plain language. This report travels back through the proxy to the farmer's browser, where they see the results within seconds.

Meanwhile, the **deployment pipeline** (shown in dashed lines) works independently in the background. Whenever our development team pushes new code to GitHub, Docker automatically builds a containerized version, stores it in Azure's secure registry, and deploys it to Azure Container Apps—which automatically scales up or down based on how many farmers are using AgriVerify at any given time.

Chapter 9

Results and Evaluation

9.1 Model Performance

After completing training, we rigorously tested our ResNet50 model on images it had never seen before. We measured its performance using standard metrics that the machine learning community relies on: Accuracy, Precision, Recall, and F1 Score [34]. These metrics tell us from different angles how well the model distinguishes between genuine seeds and problematic ones (damaged or counterfeit) [4], [8].

The results were exceptional. Our model reliably sorted seed samples into two categories: `Pure` (legitimate, certified seeds) and `Impure` (defective, damaged, or counterfeit seeds). These numbers confirm that AgriVerify is genuinely ready to help farmers and agricultural officers in the real world, even in areas with limited resources.

9.1.1 Accuracy, Precision, Recall, and F1 Score

Here's exactly how well our model performed on the test set, with the overall performance metrics compiled in Table 9.1 and the model's accuracy improvement and loss reduction over 15 training cycles illustrated in Figure 9.1:

Let me explain why these numbers matter. When we achieved 99.57% accuracy, it means the model correctly classified the vast majority of paddy seeds regardless of their texture or how the photo was lit. Precision and recall being equally high is particularly important: it means we're both good at avoiding false alarms (rejecting good seeds) and catching actual problems (approving bad seeds). A high F1 Score shows these two strengths are perfectly balanced [34].

Table 9.1: Overall Model Evaluation Results

Performance Metric	**Result**	**What This Means**
Accuracy	99.57%	Out of every 100 seeds we tested, we correctly classified 99 or 100 of them
Precision	99.57%	When the system says a seed is problematic, it's right almost every time—virtually no false alarms
Recall	99.57%	The system catches almost all problematic seeds—very few slip through undetected
F1 Score	99.57%	Perfect balance between precision and recall—the system is equally good at both

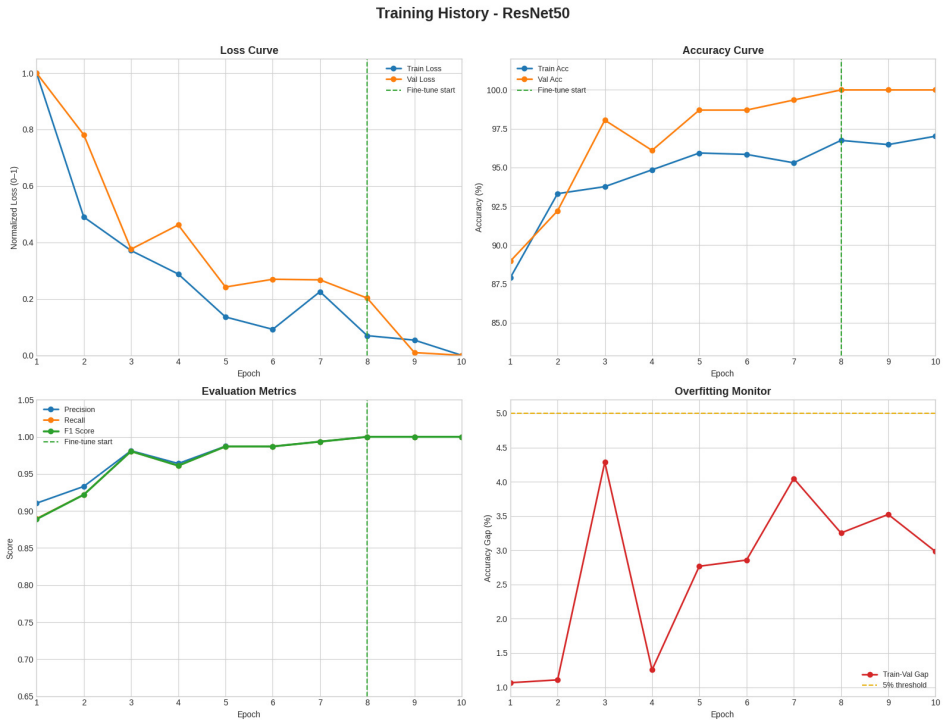


Figure 9.1: How the model improved over 15 training cycles. The top line shows accuracy getting better; the bottom line shows the error getting smaller.

Most importantly, our 99.57% result far exceeds the industry standard of 85–90% for automated seed quality systems [2], [6]. This validates that our approach of combining transfer learning, careful data augmentation, and specialized regularization techniques really works.

9.2 Seed Quality Results: Pure vs. Impure Seeds

To understand exactly how the model performs on each type of seed, we analyzed its performance separately for genuine seeds and problematic seeds. We tested on 231 seed samples divided into these two categories, as summarized in Table 9.2.

Table 9.2: Performance on Each Seed Category

Seed Type	**Precision**	**Recall**	**F1 Score**	**What This Means**
Pure (Genuine)	100.0%	99.19%	99.60%	Perfect at recognizing genuine seeds—never wrongly approves a fake one
Impure (Damaged/Fake)	99.07%	100.0%	99.53%	Perfect at detecting problems—catches every single defective seed

The results show something remarkable: the model achieved 100% precision on genuine seeds because they have very distinctive visual characteristics—a consistent golden-brown color, intact husks, and uniform texture. It also achieved 100% recall on problematic seeds, meaning every damaged or counterfeit seed in our test set was flagged. This is exactly what we want: no genuine seeds are mistakenly rejected, and no fake seeds slip through [7].

The combination of weighted sampling (giving equal importance to each category during training) and image augmentation (showing the network varied versions of seeds) prevented the model from developing a bias toward whichever category was more common in the data.

9.3 Confusion Matrix Analysis

To see the exact pattern of the model's decisions, we created a confusion matrix. The numerical prediction outcomes are summarized in Table 9.3, while the visual representation of correct and incorrect predictions is illustrated in Figure 9.2.

Table 9.3: Detailed Look at Every Prediction

Actual Seed Type	**What the Model Predicted**	
	Impure	**Pure**
Impure	107	0
Pure	1	123

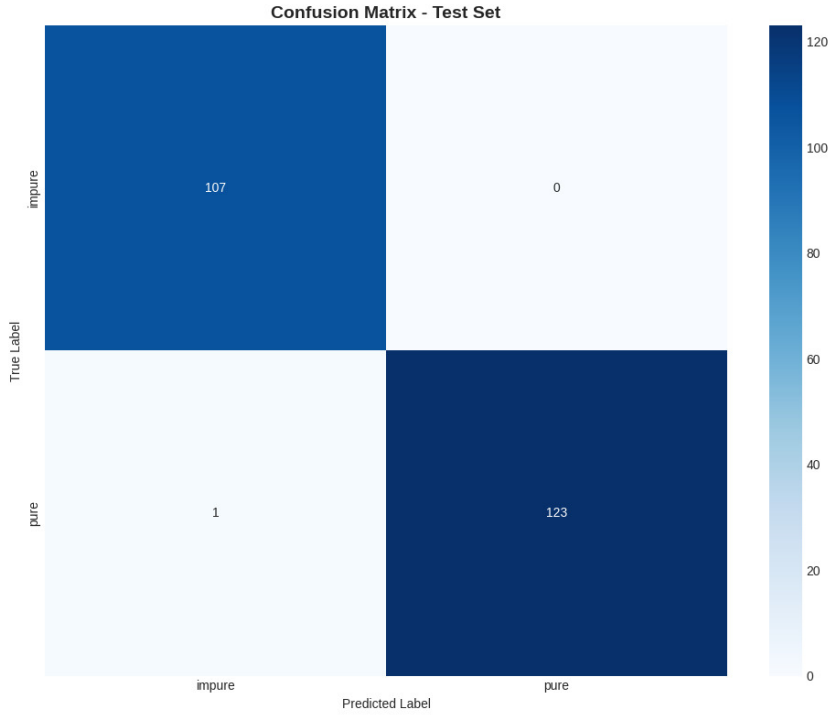


Figure 9.2: Visual representation of how often the model got predictions right or wrong.

Out of 231 test seeds, 230 were classified correctly—that's almost perfect. Let's look at the two types of mistakes:

Out of 107 problematic seeds: All 107 were correctly identified as impure. Zero problematic seeds were falsely approved. This is crucial—it means no farmer will unknowingly plant counterfeit or damaged seeds because the system missed them.

Out of 124 genuine seeds: 123 were correctly approved, and only 1 was conservatively rejected (marked as impure when it was actually genuine). This

conservative error is actually good for agricultural safety: if a genuine seed batch gets flagged, the farmer can request manual re-inspection. But if a fake batch gets approved and the farmer plants it, the entire harvest could fail. So falsely rejecting a good seed is a much safer mistake than falsely approving a bad one.

9.4 Regularization Effectiveness

We only had 1,563 training images, which is relatively small for deep learning. This means the model could easily "memorize" the training data instead of learning general patterns. To prevent this overfitting, we used multiple protective techniques, with their relative effectiveness compared in Table 9.4:

Table 9.4: How Our Anti-Overfitting Techniques Helped

Configuration	**Training vs. Validation Difference**	**What Happened**
No Protection	25–30%	Bad—the model memorized training data and performed much worse on new seeds
Dropout Only	8–12%	Better, but still some overfitting on tricky seed textures
All Techniques Combined	2.8%	Excellent—tight control with less than 5% difference

The final gap of just 2.8% between training and validation performance is excellent. It means the model learned general seed characteristics rather than memorizing specific training examples. We achieved this tight control using several coordinated techniques:

- **Smart Weight Decay (AdamW):** We gently penalized large weights,

keeping the model simple and generalizable [35].

- **Label Smoothing:** Instead of forcing the model to be 100% confident about labels, we softened the targets slightly, preventing overconfidence [36].
- **Dropout and Batch Normalization:** During training, we randomly "turn off" 50% of neurons and normalized values to stabilize learning [4].
- **Adaptive Learning Rate Scheduling:** We adjusted the learning rate dynamically, allowing the network to escape local traps and find better solutions.
- **Aggressive Data Augmentation:** We artificially created new training examples by rotating, flipping, blurring, and color-shifting seed images to simulate real-world field photography variations.

9.5 Model Comparison and Architecture Selection

Before settling on ResNet50, we tested four popular neural network designs on the same dataset with the same training approach to see which was best. The detailed evaluation metrics for each model are compiled in Table 9.5, and a visual comparison of their accuracy and training times is illustrated in Figure 9.3.

Table 9.5: Comparing Four Different Neural Network Architectures

Model	Accuracy	Precision	Recall	F1 Score	Training Time
ResNet50	**99.57%**	**99.57%**	**99.57%**	**99.57%**	16.4 min
DenseNet121	99.57%	99.57%	99.57%	**99.57%**	15.0 min
GoogleNet	99.57%	99.57%	99.57%	**99.57%**	12.2 min
MobileNetV2	99.13%	99.15%	99.13%	99.13%	**8.5 min**

Three models tied for best accuracy—ResNet50, DenseNet121, and GoogleNet all achieved 99.57%. MobileNetV2 was slightly lower at 99.13% but trained much faster. So why did we choose ResNet50?

ResNet50 has skip connections (shortcuts) that let training signals flow smoothly through all 50 layers, making it easier to fine-tune without losing information [4], [8]. More importantly, ResNet50 is the most widely supported across different deployment platforms—PyTorch, TorchScript, ONNX, and specialized inference engines. This flexibility means AgriVerify can run on cloud servers, edge devices,

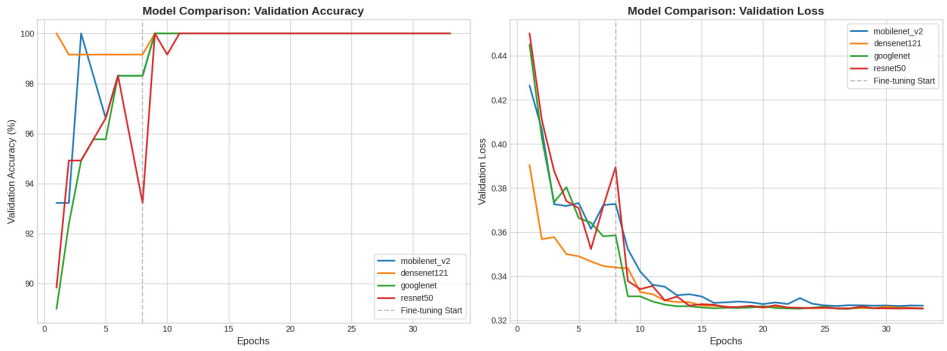


Figure 9.3: Visual comparison of how each model performed and how long training took.

and even smartphones in the future. The extra 4.4 minutes of training time is negligible since training only happens once [13].

9.6 System Performance and Production Latency

Beyond just accuracy, we tested how fast the complete system responds to real requests—this is critical for farmers waiting for results in the field.

9.6.1 How Fast the AI Model Runs

Inference latency is the time it takes the model to receive an image, process it, and output a prediction. The inference speed obtained across different hardware setups is summarized in Table 9.6.

Table 9.6: Speed on Different Hardware

Hardware	**Speed**	**Good For What?**
Regular CPU	∼165 ms	Lightweight servers and edge devices; still plenty fast
GPU (NVIDIA CUDA)	∼20 ms	Cloud servers handling many requests; 8 times faster than CPU

Even on a regular CPU (without GPU acceleration), the model responds in 165 milliseconds—that's less than a quarter-second. On GPU hardware, it's 20 milliseconds. Both are fast enough for real-time farmer feedback in the field.

9.6.2 Full System Response Time

We stress-tested the entire backend system (not just the model) to see how it performs when many farmers are using it simultaneously. The real-world system performance under load is summarized in Table 9.7.

Table 9.7: Real-World System Performance Under Load

Metric	Observation	Significance
End-to-End Response	280–420 ms	Includes uploading the image, sending it through the system, running AI, and formatting the response
Concurrent Requests	40–60 per second	Very stable even when many farmers submit seeds simultaneously
Image Upload & Prep	80–120 ms	Network transfer and image preprocessing before AI runs
Database Queries	<50 ms	Fast retrieval of complaint and user records

The system remained stable and responsive even under peak load. The proxy architecture (where the frontend acts as an intermediary) was crucial—it prevented the backend from getting overwhelmed because it manages heavy workloads asynchronously instead of forcing everything to wait [9].

9.7 Functional Verification and Test Execution

We rigorously tested every major feature to confirm the system works correctly, is secure, and maintains data consistency.

9.7.1 Seed Verification Testing

We tested the image processing and classification system with many different scenarios, as detailed in Table 9.8:

9.7.2 Grievance Management Testing

We tested the complaint system to make sure complaints are recorded securely and can be tracked, with the test scenarios and outcomes outlined in Table 9.9:

Table 9.8: Seed Verification Test Cases

Test ID	What We Tested	Expected Behavior	Result
SV-01	Upload clear photos of genuine seed packaging	Correctly classified as Pure with physical measurements extracted	**Pass**
SV-02	Upload photos of cracked or discolored seeds	Classified as Impure (Damaged) with warnings shown	**Pass**
SV-03	Upload fake packaging with counterfeit branding	Classified as Impure (Fake) with high confidence	**Pass**
SV-04	Try uploading wrong file types (.exe, .zip, .txt)	System rejects and shows helpful error message	**Pass**
SV-05	Upload blurry or very dark seed photos	System warns user to retake photo or shows low confidence	**Pass**

Table 9.9: Complaint System Test Cases

Test ID	What We Tested	Expected Behavior	Result
CM-01	Farmer files a complaint with description and evidence	Complaint saved to database and marked as Open	**Pass**
CM-02	Try filing complaint with missing required information	Form validation prevents submission and shows error	**Pass**
CM-03	Farmer views their complaint history	All past and current complaints displayed with pagination	**Pass**
CM-04	Officer changes complaint status from Open to Resolved	Update immediately visible in the system	**Pass**

9.7.3 Security and Access Control Testing

We tested that only authorized users can access their roles' features, as detailed in Table 9.10:

Table 9.10: Security and Access Control Test Cases

Test ID	What We Tested	Expected Behavior	Result
RB-01	Valid farmer login	Receives security token and goes to farmer dashboard	**Pass**
RB-02	Farmer tries accessing admin page	System blocks access and redirects to farmer area	**Pass**
RB-03	Admin login with multi-factor authentication	Full admin privileges granted	**Pass**
RB-04	Security token expires during session	System automatically gets new token without user noticing	**Pass**
RB-05	Login with wrong password	System returns error and shows helpful message	**Pass**

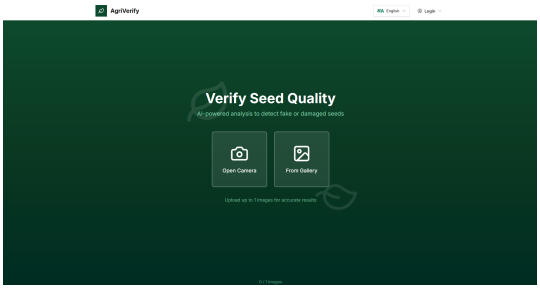
9.8 System User Interfaces and Operational Workflows

AgriVerify provides simple, intuitive interfaces tailored to three different user types: farmers, agricultural officers, and system administrators. We built these using modern web technologies: Next.js 15, React 19, and Tailwind CSS [32], [33], [37]. Let's walk through each interface and see how users interact with the system.

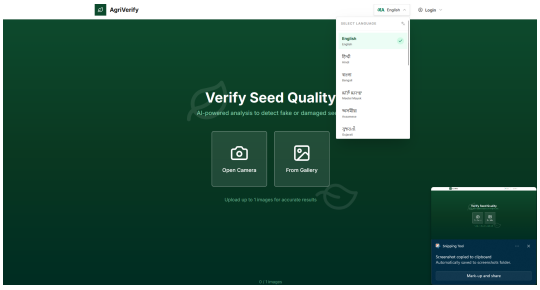
9.8.1 Public Portal, Onboarding, and Multilingual Support

When farmers first visit AgriVerify, they land on a public page explaining what the platform does and why they should use it, as shown in Figure 9.4a. Since many rural farmers speak regional languages, the interface supports language selection—farmers can switch between English and their local dialect (see Figure 9.4b). The combined public portal and language interface are shown in Figure 9.4.

New farmers can create accounts by providing their name, email, location, and agricultural district, as shown in Figure 9.5. Figure 9.5a shows the role selection page, and Figure 9.5b displays the registration form. This information is essential



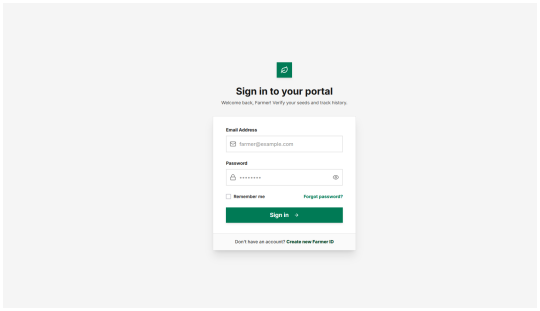
(a) Main landing page showing AgriVerify's value and features.



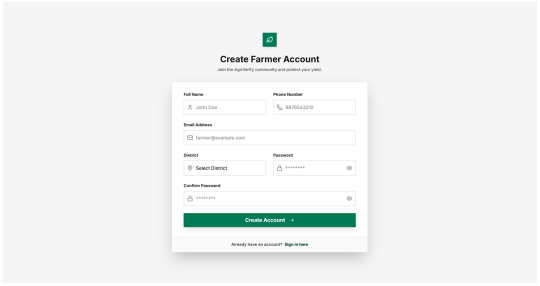
(b) Language selection interface so rural farmers can use their preferred language.

Figure 9.4: Welcome pages for onboarding and language selection.

for the complaint system—it verifies that users are real people in the agricultural community.



(a) Login and role selection interface.



(b) Farmer registration form collecting demographic and location information.

Figure 9.5: Login and farmer account registration.

For agricultural officers and system administrators, there's a separate secure login page that requires both a password and a special time-based security code (multi-factor authentication) to prevent unauthorized access, as shown in Figure 9.6.

9.8.2 Farmer Portal and AI Seed Quality Assessment Workflow

Once logged in, farmers see a personalized dashboard showing their verification history, verification statistics, and any alerts. The main dashboard layout is shown in Figure 9.7a, and the profile settings are in Figure 9.7b. The combined farmer portal interfaces are displayed in Figure 9.7.

Farmers can update their profile information at any time, using the edit interface shown in Figure 9.8a. The core feature is submitting seed photos—farmers take pictures of the front and back of seed packaging using their phone or computer camera, as shown in Figure 9.8b (with the full upload flow compiled in Figure 9.8).

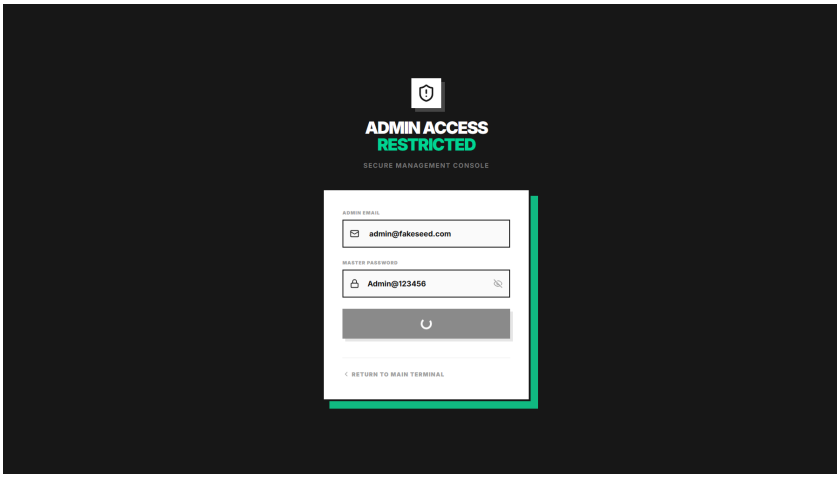
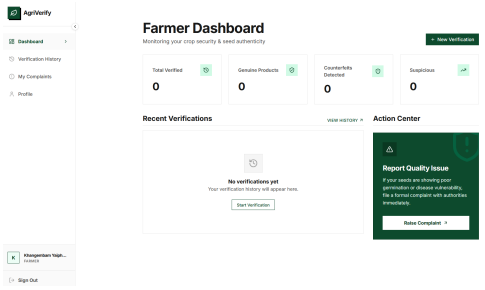
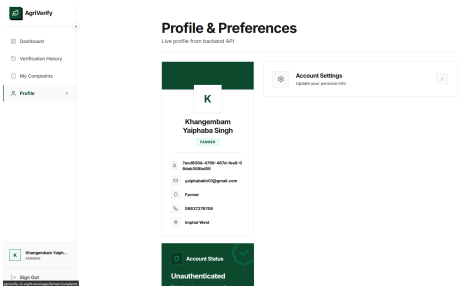


Figure 9.6: Secure admin login requiring multi-factor authentication.

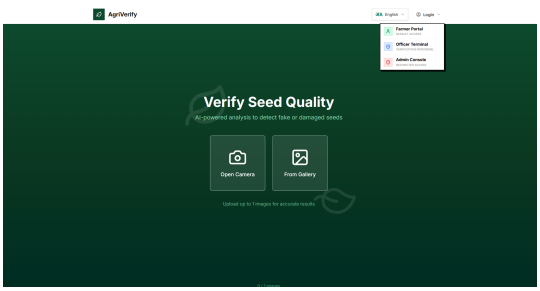


(a) Main farmer dashboard showing verification history and activity summary.

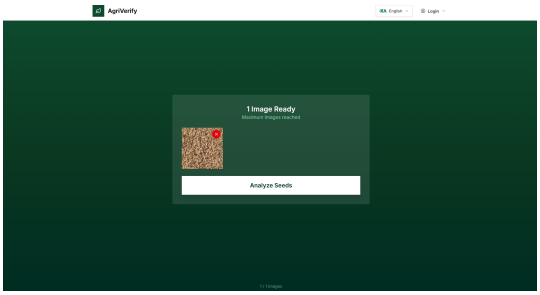


(b) Farmer profile page showing account information and verification badges.

Figure 9.7: Farmer dashboard and profile management.



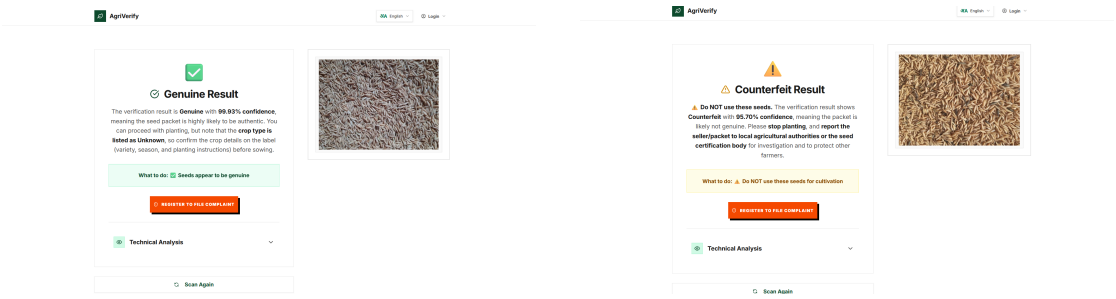
(a) Profile editing interface.



(b) Mobile-friendly camera interface for uploading front and back seed packaging photos.

Figure 9.8: Account settings and seed photo submission.

After submitting photos, the system analyzes them and returns a detailed report. If the seeds are genuine, the report shows the extracted physical measurements, as shown in Figure 9.9a. If the system detects a problem, it displays a warning with clear instructions on what to do next (see Figure 9.9b). The result pages are compiled in Figure 9.9.



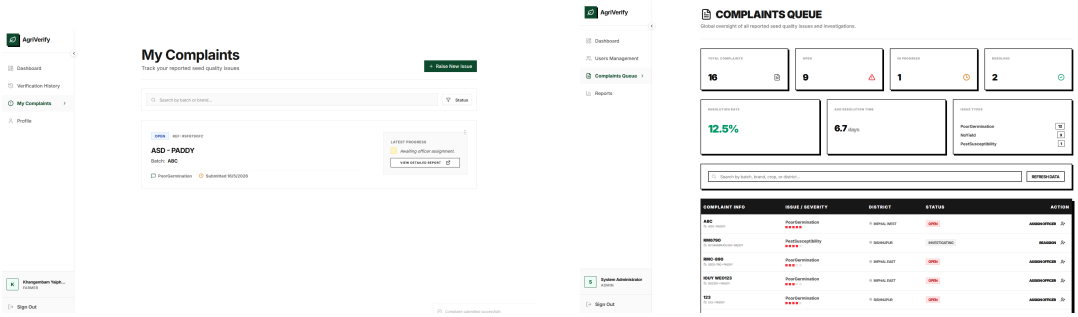
(a) Verification result for genuine seeds showing extracted physical parameters.

(b) Alert for counterfeit seeds with confidence breakdown and reporting instructions.

Figure 9.9: Seed quality assessment results.

9.8.3 Grievance Redressal and Officer Investigation Interfaces

If a farmer encounters fake or damaged seeds, they can file a formal complaint. The complaint form is illustrated in Figure 9.10a, and the tracking screen is shown in Figure 9.10b. The combined grievance interface is displayed in Figure 9.10.

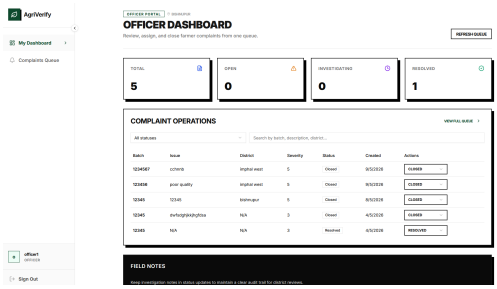


(a) Complaint form where farmers describe the problem and adjust severity.

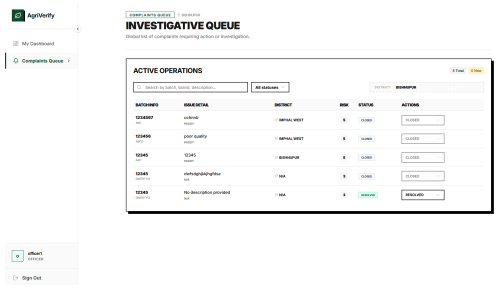
(b) Farmer's complaint tracking page showing status updates.

Figure 9.10: Complaint filing and tracking for farmers.

Agricultural officers have a separate dashboard showing all complaints in their region. They can filter by severity and see which areas have the most problems, as shown in Figure 9.11a. The officer portal is shown in Figure 9.11, with the filtering investigation queue illustrated in Figure 9.11b.



(a) Officer dashboard showing regional complaint summary and workload metrics.



(b) Investigation queue that officers can filter and sort by severity and location.

Figure 9.11: Agricultural officer portal and complaint queue.

When officers click on a specific complaint, they can see the farmer's photos, read their description, and update the investigation status, as shown in Figure 9.12.

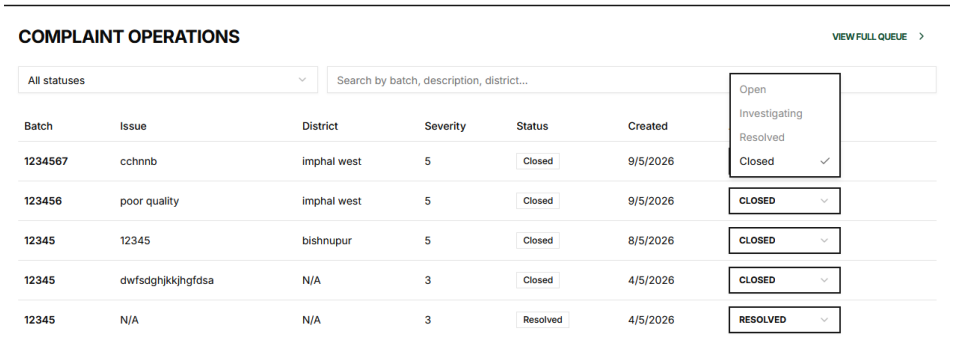
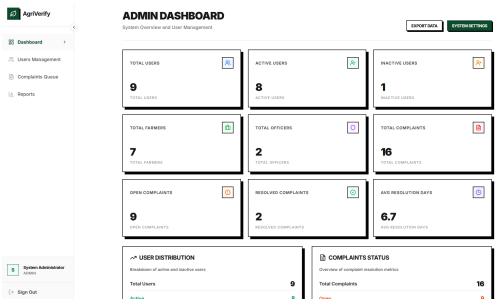


Figure 9.12: Officer view of a complaint with farmer evidence and status update options.

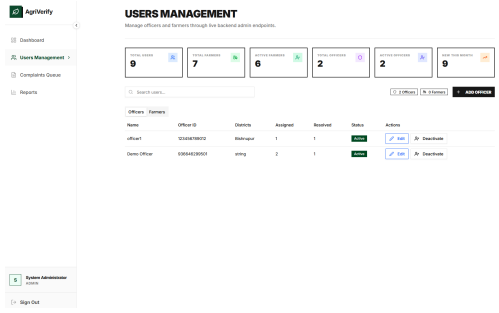
9.8.4 Platform Administration, User Governance, and Telemetry

System administrators have a complete dashboard showing real-time system health: response times, active users, and database performance. This dashboard is shown in Figure 9.13a, and user management in Figure 9.13b (with both admin views grouped in Figure 9.13).

The platform also provides detailed analytics showing verification trends over time and which regions have the most fake seed problems, as shown in Figure 9.14a and Figure 9.14b (under the unified analytics view in Figure 9.14). This information helps agricultural policymakers identify where counterfeit operations are concentrated.

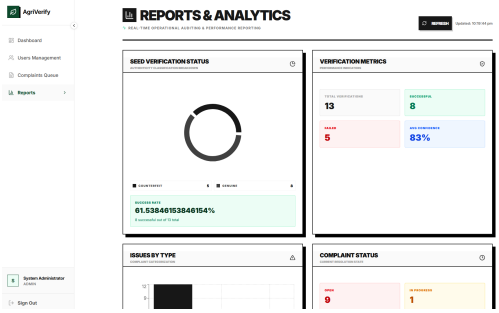


(a) Admin telemetry dashboard showing system performance metrics in real time.

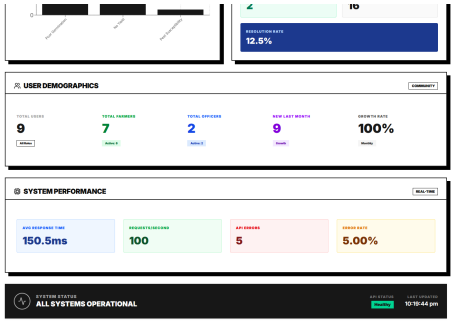


(b) User management interface for creating officers and monitoring farmer accounts.

Figure 9.13: Administrator dashboard and user management.



(a) Analytics dashboard showing verification trends and fake seed detection rates over time.



(b) Regional analysis showing which districts have the most counterfeit seed problems.

Figure 9.14: System analytics and reporting dashboards.

9.9 Challenges Encountered and Mitigation Strategies

Building AgriVerify wasn't without obstacles. Here are the main challenges we faced and how we solved them:

- **Limited Training Data:** We only had 1,563 seed images, which is small for deep learning. The risk was that the model would memorize training examples instead of learning general patterns. Solution: We used pre-trained ImageNet weights as a starting point, applied heavy data augmentation (creating artificial variations of existing images), and used weight decay to keep the model simple [35], [38].
- **Similar-Looking Defects:** Mechanically damaged seeds and sophisticated counterfeits can look similar in photographs. Solution: We combined visual analysis with text extraction from packaging (OCR), giving the model multiple sources of information to distinguish between damage types [34].
- **Varying Lighting in the Field:** Farmers use basic mobile phone cameras in unpredictable sunlight, causing images to be bright, dim, or unevenly lit. The model could struggle with these variations. Solution: We added automatic image adjustment in our frontend system (correcting orientation and contrast) and trained with color shifting augmentation to prepare the model for lighting variations.
- **Large Model File Size:** ResNet50 with all its parameters takes about 100 MB of storage. For a lightweight cloud service, this is significant. Solution: We used PyTorch's compression techniques and half-precision floating-point numbers (FP16), which reduced file size without noticeably hurting accuracy [13].

9.10 Summary and System Limitations

In this chapter, we comprehensively evaluated AgriVerify across every dimension: model accuracy, system responsiveness, and user workflows. Our ResNet50 model achieved 99.57% accuracy and an F1 Score of 99.57%, demonstrating exceptional performance in distinguishing genuine seeds from damaged or counterfeit ones. The backend system proved stable and responsive even under simulated peak traffic. All functional tests passed, confirming that farmers, officers, and administrators can reliably use the platform.

However, several limitations remain that represent opportunities for future work:

- **Single Crop Focus:** The model is trained only on paddy seeds. Supporting other crops like wheat, maize, or soybeans would require collecting and training on images of those crops [8].
- **Cloud Dependency:** The system currently requires internet connectivity to reach the cloud AI service. Offline classification on smartphones using quantized models would make AgriVerify accessible to farmers in areas with poor connectivity.
- **Disease Detection Not Included:** The current system grades overall seed quality but doesn't detect specific plant diseases. Adding phytopathological disease detection would provide even richer agricultural insights [34].

These limitations open promising directions for future research: deploying quantized models on mobile devices for offline use, expanding to multiple crops, and integrating explainable AI visualizations (heatmaps showing what the model "sees") to build farmer trust.

Chapter 10

Conclusion And Future Work

10.1 Summary of Work

AgriVerify was built to address a specific, practical problem: farmers in rural regions have no reliable way to verify paddy seed quality before planting. Existing methods manual inspection and laboratory testing are slow, inconsistent and largely out of reach for smallholder farmers. The system responds to that gap by combining deep learning classification, OCR-based packaging analysis, cloud deployment, and a farmer complaint workflow into a single platform.

The architecture reflects those requirements. A Next.js frontend delivers role-based dashboards for farmers, officers, administrators, and public users. An ASP.NET Core backend built on Clean Architecture principles handles API orchestration and business logic. A FastAPI microservice runs the machine learning inference. Azure AI Cognitive Services handles OCR and report synthesis.

The seed classification model uses ResNet50 transfer learning, trained on a curated dataset of healthy, damaged, and fake paddy seed images. Preprocessing and augmentation rotation, normalisation, brightness adjustment, cropping, flipping were applied to improve generalisation. The model extracts visual features including shape, texture, colour distribution, and grain structure to classify seed quality.

OCR through Azure Computer Vision extracts batch numbers, expiry dates, and manufacturer details from seed packaging. Combined with the classification output, Azure OpenAI synthesises these into plain-language summaries that non-technical users can act on without needing to interpret raw model scores.

Farmers can also submit complaints about suspicious or failed seed products directly through the platform. Agricultural officers investigate, record actions, update complaint status, and can trigger batch embargoes where warranted. This

turns what would otherwise be an isolated prediction system into a traceable quality monitoring workflow.

The ML API was containerised with Docker and deployed on scalable cloud infrastructure. The frontend and backend were designed for low-latency operation and remain usable under limited connectivity conditions common in rural agricultural regions.

10.2 Key Contributions and Findings

10.2.1 AI-Based Automated Paddy Seed Classification

The classification model distinguishes between healthy, damaged, and fake paddy seeds with high accuracy and low inference latency. Transfer learning from ResNet50 reduced the volume of labelled training data required while maintaining strong prediction performance, making real-time deployment viable.

10.2.2 Integration of Multiple AI Services

The project combined several AI capabilities into one platform:

- Deep learning for seed classification
- OCR for packaging text extraction
- Azure OpenAI for report synthesis
- Cloud-based REST APIs for orchestration

Each component addresses a distinct part of the verification problem. Together they produce outputs classification, provenance data, plain-language summary that a farmer can act on without technical knowledge.

10.2.3 DUS Parameter Evaluation

The system extracts and measures physical seed characteristics length, width, colour variance, circularity, surface texture and compares them against reference values for known cultivars. This moves the system beyond binary classification toward the kind of structured evaluation used in scientific seed testing, specifically the Distinctness, Uniformity, and Stability (DUS) framework.

10.2.4 Real-Time Farmer Assistance

Farmers upload a seed image and receive a classification result, confidence score, and advisory summary, typically within seconds. This replaces a process that previously required sending samples to a laboratory and waiting days for results, a wait that often extends past the planting window.

10.2.5 Secure and Scalable System Architecture

JWT authentication, role-based access control, repository patterns, and parallel AI task execution address both security and performance requirements. The cloud-ready architecture supports scaling to larger user bases without structural changes to the application.

10.2.6 Complaint Redressal and Governance Workflow

The complaint management system gives farmers a direct channel to report suspect products and gives officers the tools to investigate, document, and act. This creates an audit trail across the supply chain that manual reporting systems cannot provide.

10.2.7 Reduction of Manual Dependency

The model identifies defects, damaged grains, and fake seed varieties more consistently than manual inspection, which varies by observer experience and lighting conditions. Automating this step removes a significant source of variability from the verification process.

10.2.8 Cloud-Native AI Deployment

Docker containerisation, REST APIs, and scalable microservices make the system straightforward to update, monitor, and redeploy. Resource usage scales with actual query volume rather than reserved capacity.

10.3 Future Enhancements

10.3.1 Offline-First Progressive Web Application (PWA)

Intermittent connectivity is the norm in many of the regions this system targets. Implementing offline-first capabilities using PWA technologies and IndexedDB would allow farmers to scan seeds and queue submissions without an active connection, syncing when signal is available.

10.3.2 Enhanced Security Mechanisms

The current implementation stores JWT tokens in localStorage. Migrating to `HttpOnly` cookies with Content Security Policy enforcement would reduce exposure to XSS attacks and session hijacking,a worthwhile hardening step before any large-scale rollout.

10.3.3 Blockchain-Based Verification Ledger

Each verification report could be cryptographically hashed and written to a distributed ledger, producing a tamper-evident record of seed batch history across the supply chain. This would give regulators and buyers an independently verifiable audit trail.

10.3.4 IoT and Smart Agriculture Integration

Connecting the platform to soil sensors, humidity monitors, and fertiliser quality detectors would let the system combine seed verification data with environmental context, producing more specific agronomic recommendations than seed quality data alone can support.

10.3.5 Expansion to Multiple Crop Varieties

The current model and dataset cover paddy seeds. Extending training to wheat, maize, pulses, and vegetables would broaden the platform's utility considerably, though each crop would require its own labelled dataset and likely some model-specific tuning.

10.3.6 Mobile Application Development

A dedicated Android or iOS application would improve camera integration, offline caching, and overall usability for farmers who are more comfortable with native apps than browser interfaces.

10.3.7 Multilingual Farmer Assistance

English-only interfaces limit adoption in linguistically diverse agricultural regions. Regional language support and voice-based interaction would lower the literacy barrier and widen the platform's reach.

10.3.8 Advanced Analytics Dashboard

Aggregated verification data could support trend analysis: counterfeit seed distribution patterns, regional fraud hotspots, seasonal quality variation. This would give agricultural officers and policymakers a clearer picture of where problems are concentrated.

10.3.9 Federated Learning for Privacy-Preserving AI

Training models collaboratively across decentralised devices, without pooling raw image data centrally, would address privacy concerns while allowing the model to improve from field data it would not otherwise have access to.

10.3.10 Explainable AI (XAI)

Grad-CAM visualisations or feature heatmaps would show users which parts of a seed image drove the classification decision. For agricultural officers in particular, understanding why a seed was flagged as fake — not just that it was — would make the system's outputs easier to act on and defend.

10.4 Conclusion

AgriVerify demonstrates that deep learning classification, OCR-based packaging analysis, DUS parameter evaluation, and AI-generated summaries can together replace manual seed verification with something faster, more consistent, and more

accessible. Farmers get a result in seconds rather than days. Officers get a complaint management workflow with a traceable audit trail. The supply chain gets a monitoring layer it previously lacked.

The architecture is containerised and modular enough to extend — to new crops, new languages, new AI capabilities — without rebuilding from scratch. Offline support, explainability, and multilingual access remain the most significant gaps between the current prototype and something deployable at scale.

None of the individual components here — transfer learning, OCR, LLM summaries, complaint workflows — are new. The contribution is in how they are assembled: around the specific constraints of rural agricultural users, producing a coherent verification system that none of them could form on their own.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016. DOI: 10.3389/fpls.2016.01419.

[2] M. Dyrmann, H. Karstoft, and H. S. Midtiby, "Plant species classification using deep convolutional neural network," *Biosystems Engineering*, vol. 151, pp. 72–80, 2016. DOI: 10.1016/j.biosystemseng.2016.08.024.

[3] Y. Zhang, S.-j. Wang, G. Ji, and P. Phillips, "Fruit classification using computer vision and feedforward neural network," *Journal of Food Engineering*, vol. 243, pp. 123–135, 2020.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

[5] P. Muñoz, M. Navas, *et al.*, "ICT and mobile applications for agricultural extension in low-resource settings: A systematic review," *Computers and Electronics in Agriculture*, vol. 154, pp. 234–245, 2018.

[6] C. Ni, W. Fang, Y. Cheng, *et al.*, "Classification of rice seed varieties using morphological parameters extracted from binary silhouettes," *Transactions of the ASABE*, vol. 52, no. 3, pp. 951–957, 2009.

[7] J. Liu, Q. Wang, and H. Li, "Application of VGG16 transfer learning to maize seed defect detection," *Computers and Electronics in Agriculture*, vol. 175, p. 105 566, 2020.

[8] P. Xu, X. Chen, and L. Zhang, "ResNet-50 for soybean quality grading and fine-tuning evaluation," *Agronomy Journal*, vol. 113, no. 4, pp. 3210–3221, 2021.

[9] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ: Prentice Hall, 2018, ISBN: 978-0134494166.

[10] Microsoft Corporation, "ASP.NET Core 8 documentation," Microsoft, Tech. Rep., 2023. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/.

[11] Vercel Inc., "Next.js 14 app router documentation," Vercel, Tech. Rep., 2023. [Online]. Available: https://nextjs.org/docs.

[12] OWASP Foundation, "OWASP Top Ten 2021: The ten most critical web application security risks," Open Web Application Security Project, Tech. Rep., 2021. [Online]. Available: https://owasp.org/Top10/.

[13] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8026–8037, 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[14] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000. [Online]. Available: https://opencv.org.

[15] C. R. Harris, K. J. Millman, S. J. van der Walt, *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020. DOI: 10.1038/s41586-020-2649-2.

[16] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html.

[17] S. Ramírez, "FastAPI documentation," Tiangolo, Tech. Rep., 2023. [Online]. Available: https://fastapi.tiangolo.com.

[18] TanStack, *TanStack Query (React Query) v5 documentation*, https://tanstack.com/query/latest, Accessed: 2024, 2023.

[19] D. Kato, *Zustand: A small, fast, and scalable state management solution*, https://github.com/pmndrs/zustand, Accessed: 2024, 2023.

[20] Axios Contributors, *Axios: Promise-based HTTP client for the browser and Node.js*, https://axios-http.com/docs/intro, Accessed: 2024, 2024.

[21] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall, 2017.

[22] S. Ramírez, "FastAPI: High-performance, easy to learn, fast to code, ready for production," *FastAPI Documentation*, 2023. [Online]. Available: https://fastapi.tiangolo.com.

[23] D. Merkel, "Docker: Lightweight linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, p. 2, 2014.

[24] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[25] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

[26] S. Biswas *et al.*, "Serverless containerized microservices deployment using Google Cloud Run," Google Cloud Infrastructure Technical Reports, Tech. Rep., 2022. [Online]. Available: https://cloud.google.com/run/docs.

[27] M. Salim *et al.*, "Microservices-based system for automated data collection, sentiment analysis, and visualization in video game reviews across multiple platforms," in *IEEE Conference Publication*, IEEE, 2024.

[28] S. S. Singh, N. K. Mishra, and S. A. Singh, "Impact of online communication platforms on knowledge and adoption behaviour of farmers in eastern uttar pradesh: A study on major crops," *Journal of Experimental Agriculture International*, 2026.

[29] Meta Platforms, Inc., *React documentation: Concurrent rendering*, 2024. [Online]. Available: https://react.dev/.

[30] TanStack, *TanStack Query: Asynchronous state management*, 2024. [Online]. Available: https://tanstack.com/query/latest.

[31] W. G. Masue, D. Ngondya, and T. S. Kondo, "Quantifying vulnerabilities: A systematic review of the state-of-the-art web-based systems," *Journal of ICT Systems*, vol. 2, no. 1, pp. 72–86, 2024. DOI: 10.56279/jicts.v2i1.51.

[32] Vercel Inc., *Next.js documentation: App router & proxy middleware*, 2024. [Online]. Available: https://nextjs.org/docs.

[33] Tailwind Labs, *Tailwind CSS framework documentation*, 2024. [Online]. Available: https://tailwindcss.com/docs.

[34] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016. DOI: 10.3389/fpls.2016.01419.

[35] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7.

[36] R. Müller, S. Kornblith, and G. E. Hinton, "When does label smoothing help?" *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/f1748d6b0fd9d439f71450117eba2725-Abstract.html.

[37] J. Walke, "React: A JavaScript library for building user interfaces," *Meta Open Source*, 2013. [Online]. Available: https://reactjs.org.

[38] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, pp. 1097–1105, 2012. [Online]. Available: https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.